

QA Interview Preparation Guide

Manual · Automation · API · Mobile · Performance · AI · Coding · HR — v2
expanded

Target experience: **~2 years QA**

Market: Indian product & service companies (TCS, Infosys, Wipro, Cognizant, Accenture, Capgemini, Amazon, Flipkart, Razorpay, Zomato, Swiggy, Paytm, etc.)

Every answer is written in the voice of a candidate, with at least one example.

Edition: 2026

Contents

1. SDLC, STLC & Agile fundamentals
2. Manual testing — concepts & types
3. Test case design techniques
4. Defect / bug lifecycle
5. JIRA, test management & metrics
6. Selenium WebDriver — basics
7. Selenium — advanced & framework design
8. TestNG & JUnit
9. Cucumber / BDD
10. API testing — REST, Postman, Rest Assured
11. SQL & database testing
12. AI in QA
13. Mobile testing — concepts & scenarios
14. Appium — mobile automation
15. Load & performance testing — JMeter & JUnit
16. Playwright — basic interview questions
17. Git & GitHub for QA
18. CI/CD — Jenkins & GitHub Actions
19. Maven — build & dependency management
20. Java & OOPs — core interview questions
21. Coding interview problems with solutions
22. Scenario / situational testing questions
23. Behavioural & HR questions
24. Salary, notice period & negotiation
25. Questions YOU should ask the interviewer
26. Last-mile checklist

Each Q&A is written so you can **say it aloud in an interview** — natural sentences, not textbook lines — with at least one concrete example. The orange boxes tell you *what the interviewer is actually testing* with that question.

1. SDLC, STLC & Agile Fundamentals

Q. Walk me through SDLC.

A. SDLC is the structured lifecycle every software product follows from idea to retirement. In my projects we typically go through: requirement gathering by the BA, planning and feasibility, high- and low-level design, development, testing, deployment, and maintenance. As QA I'm involved well before the build is ready — I read requirements during the design phase so my test cases are ready by the time code lands in the QA environment.

Example: In my last sprint we had a "Forgot Password" story. The BA finalised AC on Monday, devs started coding Tuesday, and I had test cases reviewed by EOD Tuesday — so when the build was ready Thursday, we lost zero time on test design.

What they're really asking: Whether you understand QA isn't an end-of-cycle activity but runs in parallel.

Q. What's the difference between SDLC and STLC?

A. SDLC is the umbrella lifecycle for the whole product, STLC is the testing-focused lifecycle that runs in parallel. STLC has six phases — requirement analysis from a testing lens, test planning, test design, environment setup, execution, and closure. The two aren't sequential — STLC starts the moment requirements are clear, not after dev is done.

Example: When a new feature comes in, I start STLC at "requirement analysis" — I list ambiguities, edge cases, and missing AC. By the time devs are coding, my test cases are ready for review.

Q. Explain Agile and Scrum ceremonies.

A. Agile delivers software in short, iterative cycles. Scrum is the most common Agile framework — we work in 2-week sprints, with a Product Owner, Scrum Master, and a cross-functional team. Daily ceremonies I attend: stand-up (15 min — yesterday/today/blockers), backlog refinement (clarify upcoming stories), sprint planning (commit to sprint scope), sprint review/demo (show work to stakeholders), and retrospective (what went well, what didn't, action items).

Example: In our last retro I raised that test environment was unstable on Mondays after weekend deploys. We agreed devs would do a smoke test before handing the build to QA — defect count dropped the next sprint.

What they're really asking: Whether you've actually worked in Scrum or just heard the words.

Q. What is a user story and what are acceptance criteria?

A. A user story is a small, user-focused requirement written as: "As a <role>, I want <feature> so that <benefit>." Acceptance criteria are the concrete, testable conditions that have to be true for it to be "done" — usually in Given/When/Then or checklist form. AC drive my test cases directly.

Example: Story: "As a customer I want to reset my password via email link so that I can recover my account." AC: link expires in 15 minutes; new password ≥ 8 chars with 1 digit and 1 special; old password is invalidated immediately; user is auto-logged in after reset. Each AC line maps to at least one test case in my test plan.

Q. DoR vs DoD?

A. DoR (Definition of Ready) is the checklist a story must pass before being pulled into a sprint — clear AC, dependencies known, estimable. DoD (Definition of Done) is what makes a story shippable — code reviewed, unit + automation tests green, deployed to QA, no critical defects, demo'd.

Example: On my team, DoD includes "regression suite passes in CI" — so even after manual QA passes, a story isn't done until automation confirms nothing else broke.

Q. Have you worked on V-Model?

A. V-Model maps every development phase to a corresponding test phase — Requirements $\leftrightarrow$ Acceptance Testing, HLD $\leftrightarrow$ System Testing, LLD $\leftrightarrow$ Integration Testing, Coding $\leftrightarrow$ Unit Testing. The point is you plan the tests at the same time as the design, not after coding. I worked partly in V-Model on a banking client where the SDLC was Waterfall.

Q. Difference between Waterfall and Agile?

A. Waterfall is sequential — you finish one phase before the next, requirements are frozen upfront, releases are infrequent. Agile is iterative — small batches, evolving requirements, frequent releases. Waterfall suits regulated, well-known domains (defence, banking compliance). Agile suits products where the market or requirements change often, which is most modern web/mobile work.

Example: My current company is Agile — 2-week sprints. A friend at a defence PSU works Waterfall — a release cycle there is 9 months.

2. Manual Testing — Concepts & Types

Q. Verification vs Validation — with an example.

A. Verification asks "are we building the product right?" — it's static, done through reviews, walk-throughs, and inspections of documents, design, code. Validation asks "are we building the right product?" — it's dynamic, done by executing the software against user needs.

Example: When the dev writes a function to calculate GST, verification is the code review confirming the formula matches the spec. Validation is when I run the checkout flow with real cart items and confirm the displayed GST matches what the customer expects to pay.

Q. Functional vs Non-functional testing.

A. Functional testing checks *what* the system does — login works, the cart adds items, the API returns the right JSON. Non-functional checks *how* it behaves — speed, security, usability, scalability. Both types ship together in a release.

Example: For our checkout: functional = "applying coupon CODE20 reduces total by 20%." Non-functional = "checkout page loads in < 2s with 500 concurrent users."

Q. Smoke vs Sanity vs Regression — when do you use each?

A. Smoke is a shallow, wide check — is the build stable enough to test at all? It runs first on every new build. Sanity is a narrow, deep check on a specific fix or feature. Regression is a broad re-test of existing functionality after any change.

Example: Yesterday we got build 2.4.0. I ran the smoke suite (login, search, add to cart, place order — 8 cases, 4 min) — passed. Then I ran sanity on the new "saved addresses" feature. Before release on Friday I'll run the full regression — about 200 automated + 30 manual cases.

Q. What is exploratory testing? Have you done it?

A. Exploratory testing is simultaneous learning, test design, and execution — no pre-written cases. I use it when documentation is poor, when I'm new to a module, or after I've finished scripted testing to hunt for unknowns. I pair it with session-based notes so I can show what I covered.

Example: When we onboarded a third-party shipping integration, the API docs were vague. I spent two 90-minute exploratory sessions hitting it with weird inputs — empty pincodes, special characters, oversized weights. Found three issues no scripted test would have caught.

Q. Black-box vs White-box vs Grey-box?

A. Black-box: I test through the UI or API without seeing code — what most QA does. White-box: I test code paths, branches, statements — that's usually unit testing. Grey-box: I have partial knowledge — for example, I know the DB schema so I can design smarter data, or I read logs to verify what's happening behind the scenes.

Example: For the order service I'm grey-box — I test through the API, but I also check the orders table and Kafka topic after each test so I catch silent failures.

Q. Difference between unit, integration, system, and acceptance testing.

A. Unit is at function/class level, done by developers. Integration tests how two or more modules talk to each other — typically API-level. System tests the entire integrated product end-to-end. Acceptance is done by the business or end users to confirm it meets their needs (UAT). The pyramid says many unit, fewer integration, even fewer system, very few acceptance.

Example: For "place order": unit = price-calculation function; integration = order-service ↔ payment-service; system = full UI checkout; acceptance = the business team running checkout against their real-world scenarios before sign-off.

Q. Alpha vs Beta testing?

A. Alpha is internal — done by internal employees in a controlled environment before public release. Beta is external — a small slice of real users in the real environment, before general availability. Beta surfaces bugs that only show up in messy real-world conditions like flaky networks or unusual devices.

Example: When we rolled out our new payment flow, we did 1 week of alpha with internal employees, then 2 weeks of beta with 5% of users behind a feature flag, then 100% rollout.

Q. What is retesting and how is it different from regression?

A. Retesting is specifically running the failed test again on the new build to confirm the defect is fixed. Regression is broader — running other tests to make sure the fix didn't break anything else. Retesting is targeted; regression is speculative.

Example: A dev fixed a defect where coupons over 50% off weren't applying. Retesting: I rerun that exact case. Regression: I rerun all coupon-related cases — fixed-amount coupons, expired coupons, first-time-user coupons — because the fix touched shared discount logic.

Q. Performance, load, stress, spike, soak — which have you done?

A. Performance testing measures speed and responsiveness under normal load. Load tests behaviour at expected peak. Stress pushes beyond capacity to find the breaking point. Spike simulates sudden surges. Soak is sustained load over hours/days to catch memory leaks.

Example: For our last release I ran a load test in JMeter — 500 concurrent users for 30 minutes against the search API. We hit 95th percentile of 1.2s, well under the 2s SLA. Then I ran a spike — jumped from 50 to 1500 users in 30s — and caught a connection-pool exhaustion bug.

Q. Usability vs Accessibility testing?

A. Usability is whether the product is intuitive and pleasant for any user. Accessibility is whether users with disabilities can use it — keyboard navigation, screen-reader compatibility, colour contrast, proper ARIA labels, WCAG compliance.

Example: During an accessibility pass I tab-navigated our checkout — I found that the "Apply Coupon" button was reachable but its focus state was invisible (contrast too low). Filed a defect with the WCAG 2.1 reference.

Q. What is compatibility / cross-browser testing?

A. Verifying the app works across combinations of browser, OS, device, screen size, and network. I use BrowserStack and LambdaTest for matrix coverage, plus a few real devices in the office.

Example: Last release we caught a Safari-only bug: the date picker didn't honour min attribute on iOS. Chrome and Edge were fine. Without cross-browser testing it would have shipped.

Q. What is risk-based testing?

A. Prioritising tests by risk = likelihood × impact. Critical, business-impacting flows get heavy coverage; low-risk areas get less. It's how I decide what to cut when timelines tighten.

Example: When two days were cut from our last sprint, I dropped the "settings page UI polish" tests and kept everything on checkout and payment. Manager agreed and we shipped with known minor cosmetic risk documented.

Q. Explain the test pyramid.

A. It's a guideline for test distribution: many fast unit tests at the base, fewer API/integration tests in the middle, very few slow UI tests at the top. Inverted ("ice-cream cone") is an anti-pattern — heavy reliance on slow UI tests, with weak unit coverage. UI tests are slow and flaky; unit tests are fast and stable, so most of your safety net should live there.

Example: Our suite is roughly 1500 unit, 400 API (Rest Assured), 90 UI (Selenium). The UI tests cover only critical user journeys.

3. Test Case Design Techniques

Q. Test scenario vs test case vs test script — with example.

A. A scenario is the high-level "what to test." A test case is the detailed steps, data, expected vs actual. A script is the automation code that runs the case.

Example: Scenario: "Verify user can log in with valid credentials." Test case: ID TC_LOGIN_001, steps 1–4, expected "redirect to /dashboard". Script: `LoginTest.loginWithValidCredentials()` in Selenium.

Q. What does a strong test case contain?

A. ID, title, preconditions, test data, steps (numbered), expected result, actual result, status, priority, requirement reference, environment, tester, and date. Anyone on the team should be able to execute it without asking me a question.

Q. Equivalence Partitioning — explain with an example.

A. Divide inputs into classes where the system is expected to behave the same, and test one value per class. It reduces redundancy — you don't test ten ages between 18 and 25 if they all hit the same code path.

Example: Age field accepts 18–60. Classes: <18 invalid, 18–60 valid, >60 invalid. So one test each at age 10, 30, 70 — instead of 50 cases.

Q. Boundary Value Analysis.

A. Defects cluster at the edges of input ranges, so we explicitly test the boundaries plus one inside and one outside. Combined with EP, this is the most common technique I use day to day.

Example: Same age field, 18–60. I test: 17, 18, 19, 59, 60, 61. The 17 and 61 cases caught a bug where the dev used `>=` instead of `>`.

Q. Decision Table Testing.

A. A table mapping combinations of conditions to expected actions. Great for complex business rules where missing a combination is a real risk.

Example: For loan approval rules — age > 21, salary > 50k, CIBIL > 700, no existing default — I built an 8-row decision table covering every yes/no combination. Caught a bug where someone with high CIBIL but defaulted loan was still being auto-approved.

Q. State Transition Testing.

A. Model the system as states and transitions, then test valid transitions and rejected invalid ones. Useful for workflows.

Example: ATM: Idle → Card Inserted → PIN Entered → Transaction → Idle. I tested: wrong PIN three times → card retained; cancel mid-transaction → eject card; timeout at PIN screen → return to Idle.

Q. Pairwise / Orthogonal Array.

A. A combinatorial technique that covers every pair of input parameters rather than every full combination. Catches most interaction bugs with a fraction of the test count.

Example: Browser × OS × Locale = 5 × 4 × 6 = 120 combos. Pairwise tooling (PICT) reduced that to ~30 with the same pair coverage. Saved 75% effort on the device matrix.

Q. When do you use Error Guessing?

A. It's an experience-based supplement — guessing where defects are likely based on past bugs and tester intuition. Useful at the end of scripted testing.

Example: On forms I always try: empty fields, copy-paste of password into a masked field, special characters in name (O'Brien), very long inputs, double-click on submit, browser back after submit. These catch bugs that formal techniques miss.

4. Defect / Bug Lifecycle

Q. Walk me through the defect lifecycle.

A. Typical states: New → Assigned → Open → Fixed → Retest → Verified → Closed. Side branches: Rejected (not a bug), Deferred (fix later), Duplicate, Reopened. I move tickets through these states in JIRA daily.

Example: Last week I logged a defect — auto-assigned via JIRA workflow to the dev lead, picked up by a dev, marked Fixed in build 2.4.1, I retested and Verified it, then Closed. From New to Closed was two days.

Q. Severity vs Priority — with concrete examples.

A. Severity is the technical impact on the system; priority is the business urgency. They don't always agree.

Example:

Defect	Severity	Priority
Company logo on homepage misspelled	Low	High
App crashes for users on Android 7 (1% of base)	High	Low
Payments failing for all users	Critical	Critical
2px spacing off on settings page	Low	Low

Q. What goes into a good bug report?

A. A clear title, environment (build, browser, OS), preconditions, numbered steps to reproduce, expected and actual results, severity and priority, attachments (screenshot/video/HAR/console logs), reproducibility (always/intermittent), and a workaround if any. The goal: a developer should reproduce it in 60 seconds without asking me.

Example: My last defect title was "Coupon CODE20 applied as 100% discount on carts $\geq$ ₹10,000 (build 2.4.0, Chrome 130, Win 11)" — title alone tells the dev the surface area and severity.

Q. What if a developer says "not a bug"?

A. I don't argue in comments. I bring evidence — show the AC, attach the HAR file, walk through the steps. If the spec is silent, I escalate to the BA or PO for clarification, not a personal opinion war. Goal is the right product, not "winning" the JIRA ticket.

Example: In our last sprint a dev rejected a defect on the password reset link. I screen-shared, showed him the spec line "link must expire in 15 minutes," demonstrated the link still working at minute 22, and we reopened it within five minutes.

Q. When would you mark a bug "Won't Fix" or "Deferred"?

A. "Won't fix" when it's by-design, cosmetic in a deprecated area, or cost-to-fix outweighs benefit. "Deferred" when it's a real bug but lower priority than current sprint commitments — pushed to backlog with a fixVersion.

Example: We had a defect where pasted images in chat lost EXIF orientation. The feature was being removed in two sprints, so PO marked it "won't fix" with a comment.

5. JIRA, Test Management & Metrics

Q. How do you use JIRA day to day?

A. I track stories, log defects, link tests to stories via Zephyr/Xray, update statuses, attach evidence, and use filters/dashboards. JQL is my primary tool — I have saved queries for "my open defects," "current sprint defects by priority," and "defects ageing > 7 days."

Q. Show me some JQL queries you've written.

A.

```
project = ECOM AND status = "In QA" AND assignee = currentUser()
project = ECOM AND issuetype = Bug AND priority in (Highest, High) AND resolution = Unresolved
project = ECOM AND fixVersion = "2.4.0" AND status != Done
project = ECOM AND updated >= -1d ORDER BY updated DESC
project = ECOM AND created >= startOfMonth() AND issuetype = Bug
labels in (regression, ui) AND fixVersion = EMPTY
```

Q. QA metrics you've tracked or reported?

A.

- **Defect Density** = defects per module/KLOC — tells me where the riskiest code lives.
- **Defect Removal Efficiency (DRE)** = defects found in QA ÷ (QA + production) × 100 — last quarter ours was 94%.
- **Defect Leakage** = production defects ÷ total defects.
- **Test Coverage** = tested requirements ÷ total requirements.
- **Automation Coverage** = automated cases ÷ total cases — ours is around 70% for regression.
- **MTTD/MTTR** — mean time to detect/resolve.
- **Pass rate, flaky-test percentage, defect ageing.**

Example: When I reported DRE per release in our QBR, leadership noticed the auth module had the worst DRE three releases in a row — they invested in better unit tests for it.

Q. What is a Requirements Traceability Matrix (RTM)?

A. A table mapping requirements ↔ test cases ↔ defects. It proves every requirement is covered and helps impact analysis when requirements change.

Example: When the BA changed the OTP expiry from 5 to 10 minutes, I pulled the RTM, found six affected test cases and one automated script, updated them all in one sitting.

6. Selenium WebDriver — Basics

Q. What is Selenium WebDriver and what's new in Selenium 4?

A. Selenium WebDriver is an open-source library that drives a real browser through the browser's native automation interface. From Selenium 4, the protocol moved to W3C standard, the JSON Wire Protocol is gone. Key new things I use:

- **Relative locators** — `above()`, `below()`, `toLeftOf()`, `toRightOf()`, `near()`.
- **Selenium Manager** — no need for WebDriverManager or manual driver downloads.
- **New window/tab API** — `driver.switchTo().newWindow(WindowType.TAB)`.
- **CDP access** — I can mock network responses, set geolocation, capture console logs.

Example: I used relative locators to fix a flaky test:

`RelativeLocator.with(By.tagName("input")).toRightOf(By.id("label-email"))` — much more stable than the absolute XPath that kept breaking.

Q. List the Selenium locators in your priority order.

A. `id` → `name` → CSS selector → XPath, in that order. Plus class name, tag name, link text, partial link text. ID is fastest and uniquely identifying (when devs use them). CSS is fast and concise; XPath is powerful but slower — I use it when I need text matching or traversal.

Example: For our search bar I'd write `By.id("search-input")`. For "click the input next to a label," I'd write `By.xpath("//label[text()='Email']/following-sibling::input")`.

Q. CSS selector vs XPath — which do you prefer?

A. CSS where possible — faster, more concise, native browser support. XPath when I need to traverse upward (`ancestor:::`), match by text, or pick by index. I avoid absolute XPaths — they break with any DOM change.

Example: For a "Sign Up" link: `a[href='/register']` in CSS. For "the row whose first cell says 'John': `//tr[td[normalize-space()='John']]` in XPath.

Q. Implicit vs Explicit vs Fluent wait.

A. Implicit wait is set once on the driver and applies to every `findElement` — it polls until the element appears or the timeout hits. Explicit wait uses `WebDriverWait` with an `ExpectedCondition` on a specific element — best for waiting on dynamic states like clickability. Fluent is explicit with custom polling intervals and exceptions to ignore.

Example:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
wait.until(ExpectedConditions.elementToBeClickable(By.id("login"))).click();
```

What they're really asking: Whether you know to **never mix implicit and explicit waits** — they compound unpredictably.

Q. How do you handle dropdowns?

A. For native HTML `<select>` I use Selenium's `Select` class. For custom dropdowns (React/Angular divs), I click to open then click the option directly — `Select` won't work on non-select elements.

Example:

```
Select s = new Select(driver.findElement(By.id("country")));
s.selectByVisibleText("India");

// For a custom React dropdown:
driver.findElement(By.cssSelector(".country-trigger")).click();
driver.findElement(By.xpath("//li[normalize-space()='India']")).click();
```

Q. How do you handle frames and iframes?

A. Switch into the frame before interacting, switch out when done. You can switch by name, index, or `WebElement`.

```
driver.switchTo().frame("payment-frame");
// ... do work
driver.switchTo().defaultContent(); // back to main page
```

Example: Our payment widget is in an iframe. I switch in, type the card number, then explicitly switch back before trying to click "Place Order" — forgetting that line caused two days of flakiness once.

Q. How do you handle multiple windows or tabs?

A. Capture the parent handle, iterate through all handles, switch to the new one. Always switch back to parent and close children.

```
String parent = driver.getWindowHandle();
for (String h : driver.getWindowHandles()) {
    if (!h.equals(parent)) {
        driver.switchTo().window(h);
        // verify
        driver.close();
    }
}
driver.switchTo().window(parent);
```

Q. How do you handle alerts?

A.

```
Alert a = driver.switchTo().alert();
String msg = a.getText();
a.sendKeys("input for prompt");
a.accept(); // or a.dismiss();
```

For browser-native alerts only. For modal HTML dialogs, use normal locators.

Q. What is the Actions class used for?

A. For complex user gestures — hover, drag-drop, right-click, double-click, keyboard combos.

```
Actions a = new Actions(driver);
a.moveToElement(menu).click(submenu).perform();
a.dragAndDrop(src, dst).perform();
a.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();
```

Example: For a mega-menu where hover reveals sub-items, regular click didn't work. `moveToElement(menu).click(subItem).perform()` fixed it.

Q. When do you use JavaScriptExecutor?

A. As a fallback when WebDriver's regular actions don't work — scrolling, clicking hidden elements, reading JS variables, setting values directly. I use it sparingly because it bypasses real user interaction and can mask UX bugs.

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].scrollIntoView(true);", el);
js.executeScript("arguments[0].click();", el);
String title = (String) js.executeScript("return document.title;");
```

Example: A footer link was below the fold and a sticky cookie banner intercepted clicks. `js.executeScript("arguments[0].click();", el)` was a temporary workaround — but I filed a defect because real users would also struggle to click it.

Q. Common Selenium exceptions and how you handle them.

A.

- **NoSuchElementException** — locator didn't match. Fix locator or add a wait.
- **StaleElementReferenceException** — the DOM re-rendered. Re-locate the element inside a try/retry.
- **ElementClickInterceptedException** — another element (overlay, modal) is on top. Wait or scroll into view.
- **ElementNotInteractableException** — present but not yet clickable. Wait for `elementToBeClickable`.
- **TimeoutException** — wait expired. Increase timeout or fix underlying issue.
- **SessionNotCreatedException** — driver/browser version mismatch. Update via Selenium Manager.

Example: In our React app I kept hitting StaleElement on a filter chip after the list refreshed. I wrapped the click in a 3-attempt retry that re-finds the element each time — flakiness dropped to zero.

Q. close() vs quit()?

A. close() closes the current window only; if it's the last, the session ends. quit() ends the whole WebDriver session and closes all windows. In tearDown I always use quit().

Q. How do you handle file upload?

A. For `<input type="file">` I just sendKeys the full path:
`element.sendKeys("C:/data/resume.pdf");`. For custom drag-drop widgets or OS file pickers, I use either AutoIT, Robot, or — better — bypass the UI and call the upload API directly.

7. Selenium — Advanced & Framework Design

Q. What is the Page Object Model and why use it?

A. POM is a design pattern where every page is a class containing its locators and operations. Tests call methods on these page objects rather than touching driver. Benefits: readable tests, reusable logic, and selector changes happen in one file.

Example: Our LoginPage class has methods `enterEmail()`, `enterPassword()`, `submit()`, and `loginAs(email, pwd)`. The test reads as `new LoginPage().loginAs("a@b.com", "x")` — nothing about CSS or XPath.

Q. POM vs Page Factory?

A. Page Factory uses `@FindBy` with lazy-initialised proxies. It's elegant but its built-in `StaleElement` handling is rough, and explicit waits don't compose as cleanly. Most modern frameworks use plain POM with explicit waits.

Q. Types of automation frameworks you've worked with.

A.

- **Data-driven** — same script, multiple datasets (TestNG `DataProvider`, Excel, JSON).
- **Keyword-driven** — actions abstracted into keywords; non-technical testers can write tests in spreadsheets.
- **Hybrid** — combines data-driven + keyword + POM. This is what we use.
- **BDD** — Cucumber with Gherkin features. Used for stakeholder-readable scenarios.

Example: Our framework is hybrid: POM for pages, TestNG `DataProvider` for data variation, Cucumber on top for critical user journeys that product reviews monthly.

Q. If asked to design a framework from scratch, walk me through your structure.

A.

```
src/  
  main/java/  
    config/      ConfigReader (env vars override properties)  
    driver/      WebDriverFactory, DriverManager (ThreadLocal)  
    pages/       BasePage + one class per page  
    utils/       DataGenerator, JSON/Excel readers, API helpers  
    reporting/   ExtentManager, screenshot helper  
  test/java/  
    tests/       BaseTest + test classes  
    listeners/   TestNG listener for Extent + screenshots  
    runners/     CucumberRunner (if BDD)  
    stepdefs/    Step definitions  
  test/resources/
```

```
config.properties
testdata.json
features/      *.feature
```

Plus Maven (or Gradle), TestNG, Cucumber, Extent, SLF4J, and a CI pipeline (Jenkins or GitHub Actions) that runs headless on every PR.

Q. How do you achieve parallel execution in Selenium?

A. Wrap WebDriver in a `ThreadLocal<WebDriver>` so every thread gets its own session. In `testng.xml` set `parallel="methods"` and `thread-count="N"`. For scaling beyond a single machine I run on Selenium Grid or a cloud provider like BrowserStack.

Example: We went from 38 min sequential to 9 min parallel (8 threads) for our nightly regression. `ThreadLocal` was the key — each thread quits its own driver in `@AfterMethod`.

Q. What is Selenium Grid?

A. Hub-node architecture for distributed parallel execution across machines, browsers, OSes. In Selenium 4 it has standalone, hub, and fully distributed modes, with Docker and Kubernetes support.

Q. Headless execution — when is it appropriate?

A. In CI and for bulk regression runs — it's faster and doesn't need a display. Locally for debugging I run headed. Some rendering bugs only show in headed mode, and some anti-bot systems detect headless.

Q. How do you handle dynamic elements?

A.

- Stable attributes: `data-testid`, `aria-label`, `role`, `name` — these don't change between releases.
- Partial matches: `contains(@id, "userRow_")`, `starts-with()`.
- Relative XPath anchored on stable text: `//label[text()='Email']/following-sibling::input`.
- Wait for the right state — `elementToBeClickable`, not just `presenceOfElement`.

Example: Auto-generated IDs like `btn-43ab8c` were breaking my tests on every deploy. I asked devs to add `data-testid="submit-btn"`. They agreed because it cost them nothing — and our flakiness on that page dropped to zero.

8. TestNG & JUnit

Q. TestNG annotations and their execution order.

A.

```
@BeforeSuite
  @BeforeTest
    @BeforeClass
      @BeforeMethod
        @Test
          @AfterMethod
            @AfterClass
              @AfterTest
                @AfterSuite
```

Plus: @DataProvider, @Factory, @Parameters, @Listeners, @BeforeGroups, @AfterGroups.

Q. TestNG vs JUnit 5.

A.

	TestNG	JUnit 5
Suite XML	Yes — very powerful	No (uses tags)
Parallel execution	Built-in via XML	Yes via config
DataProvider	Native annotation	@ParameterizedTest
Dependencies	dependsOnMethods	Limited
Listeners	Very flexible	Extension API

TestNG is more flexible for big test suites; JUnit 5 is cleaner for modern code-first projects.

Q. DataProvider example.

A.

```
@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        {"valid@x.com", "Pass123!", true, "/dashboard"},
        {"invalid@x.com", "wrong", false, "Invalid credentials"},
        {"", "Pass123!", false, "Email required"}
    };
}

@Test(dataProvider = "loginData")
```

```
public void login(String email, String pwd, boolean ok, String expected) {  
    new LoginPage().enterEmail(email).enterPassword(pwd).submit();  
    if (ok) Assert.assertTrue(driver.getCurrentUrl().contains(expected));  
    else    Assert.assertEquals(loginPage.getError(), expected);  
}
```

Q. Soft assertions — when do you use them?

A. SoftAssert collects failures and continues, then `assertAll()` reports them at the end. I use it when checking multiple independent fields on a page — like a confirmation page with 7 displayed values — so I don't have to fix and rerun 7 times.

Example:

```
SoftAssert sa = new SoftAssert();  
sa.assertEquals(page.getOrderId(), expectedId);  
sa.assertEquals(page.getTotal(), "₹1,250");  
sa.assertEquals(page.getEmail(), "a@b.com");  
sa.assertAll();
```

Q. How do you retry a flaky test in TestNG?

A. Implement `IRetryAnalyzer`, attach it via `IAnnotationTransformer`. I cap retries at 2 and log every retry so I can see flakiness rates. Retry is a band-aid — the real fix is the root cause.

Q. Test groups and dependencies — useful or risky?

A. Useful for organising suites — `groups = {"smoke"}` and including/excluding in `testng.xml`. `dependsOnMethods` is risky because it couples tests; if A fails, B is skipped, and you've lost coverage. I prefer making each test independent and using groups for filtering.

9. Cucumber / BDD

Q. What is BDD and why use Cucumber?

A. BDD is a collaboration practice where business, devs, and QA write features and scenarios together in plain English (Gherkin), and those scenarios are executable. Cucumber lets non-technical stakeholders read and even contribute to test scenarios.

Example: During refinement, our PO and BA literally write the Given/When/Then for new stories with us. Then I add step definitions. The same file is the spec, the test, and the documentation.

Q. Anatomy of a feature file.

A.

```
Feature: User login
  As a registered user
  I want to log in
  So I can see my dashboard

Background:
  Given I am on the login page

@smoke
Scenario: Valid credentials redirect to dashboard
  When I log in with "user@x.com" and "Pass123!"
  Then I should see the dashboard page

Scenario Outline: Invalid credentials show error
  When I log in with "<email>" and "<pwd>"
  Then I should see error "<msg>"
  Examples:
    | email      | pwd      | msg                      |
    | user@x.com | wrong    | Invalid password        |
    |             | Pass123! | Email is required       |
```

Q. Cucumber hooks.

A. @Before, @After, @BeforeAll, @AfterAll, and tagged variants like @Before("@payment"). I use them for driver init/quit, seeding fresh test data, and attaching a screenshot on scenario failure.

Q. What's a DataTable and DocString?

A. DataTable: a table after a step, accessed as List<Map<String,String>> in step definitions.

DocString: a multi-line string in triple quotes. I use DataTables for bulk data setup steps like "given the following users exist."

Example:

Given the following products exist:

name	price	stock	
Mouse	500	50	
Cable	200	100	

Q. Cucumber tags — why use them?

A. To filter scenarios at runtime. We tag @smoke, @regression, @wip. CI runs @smoke on every commit, full regression nightly. I exclude @wip from any CI run so half-built scenarios don't break the pipeline.

Q. BDD anti-patterns to avoid.

A.

- UI-level Gherkin: "When I click the button with id 'login-btn'" — write the behaviour, not the click.
- Long scenarios with 15 steps — keep them 3–7.
- Writing Gherkin after coding — that defeats the collaboration purpose.
- Step-definition explosion — keep steps reusable.

10. API Testing — REST, Postman, Rest Assured

Q. What is API testing and why is it valuable?

A. API testing hits the application's APIs (REST, SOAP, GraphQL) directly, bypassing the UI. It's faster, more stable, and catches bugs earlier than UI tests. It's the middle layer of the test pyramid and where I spend roughly 40% of my automation time.

Example: A login regression that took 8 seconds via Selenium runs in 200 ms via Rest Assured. Across our suite of 400 API tests, we save almost an hour per regression.

Q. REST vs SOAP.

A.

REST	SOAP
Architectural style	Strict protocol
JSON (mostly), XML, others	XML only
HTTP only	HTTP, SMTP, TCP, others
Stateless, lightweight	Heavier, supports stateful
No built-in standards for security	WS-Security, WS-AtomicTransaction
Most modern web APIs	Enterprise, banking, legacy

Q. HTTP methods and idempotency.

A.

Method	Purpose	Idempotent?	Safe?
GET	Read	Yes	Yes
POST	Create	No	No
PUT	Replace	Yes	No
PATCH	Partial update	Generally no	No
DELETE	Remove	Yes	No
OPTIONS	Capabilities	Yes	Yes

Idempotent means calling N times has the same effect as calling once.

Q. Important HTTP status codes.

A. 2xx success (200, 201, 202, 204) · 3xx redirect (301, 302, 304) · 4xx client error (400, 401, 403, 404, 405, 409, 422, 429) · 5xx server error (500, 502, 503, 504).

Example: Last week our checkout API returned 200 with `{"error": "out of stock"}` in the body. That's an anti-pattern — should be 409 Conflict. I filed a defect because frontend was treating 200 as success.

Q. What do you validate in API testing?

A. Status code, response headers (content-type, cache, auth, rate-limit), response body (schema, field types, mandatory fields, error format), response time vs SLA, DB state if accessible, authentication and authorization, negative paths (missing fields, wrong types, oversized payload).

Q. Path param vs Query param vs Body.

A. Path param identifies a resource: `/users/{id}`. Query params filter/sort: `/users?role=admin&page=2`. Body carries the payload for POST/PUT/PATCH — usually JSON.

Q. Authentication mechanisms you've tested.

A. Basic Auth (base64), API Key in header or query, Bearer/JWT in Authorization header, OAuth 2.0 (client credentials, authorization code, refresh tokens), session cookies. For OAuth I usually call the token endpoint once and reuse the access token across tests.

Example: In Postman I set up a pre-request script that fetches a fresh JWT if the saved one is expired — every collection request uses `{{token}}` automatically.

Q. How do you use Postman?

A. Collections grouped by resource, environments for dev/qa/prod with variables, pre-request scripts for auth and dynamic data, test scripts for assertions, Collection Runner for batch runs, and Newman in CI for headless execution. I version Postman collections in git as JSON.

Q. Sample Postman test script.

A.

```
pm.test("status is 201", () => pm.response.to.have.status(201));

const json = pm.response.json();
pm.test("returns id", () => pm.expect(json.id).to.be.a("number"));
pm.test("response under 500ms",
    () => pm.expect(pm.response.responseTime).to.be.below(500));

pm.environment.set("userId", json.id); // chain into next request
```

Q. Show me a Rest Assured test.

A.

```
given()
    .baseUrl("https://api.example.com")
    .header("Authorization", "Bearer " + token)
    .contentType(ContentType.JSON)
```



```
.body(Map.of("name", "Akshat", "role", "QA"))
.when()
  .post("/users")
.then()
  .statusCode(201)
  .body("id", notNullValue())
  .body("role", equalTo("QA"))
  .time(lessThan(2000L));
```

Q. Schema validation in Rest Assured.

A.

```
.then().body(matchesJsonSchemaInClasspath("schemas/user.json"));
```

Schema acts like a contract test — catches when devs add/remove/rename fields. I save the schema in `src/test/resources/schemas/` alongside the test.

Q. How do you test a POST that creates a resource end-to-end?

A.

1. POST valid payload → assert 201, Location header, response body matches schema.
2. GET the new resource by id → assert it matches.
3. POST same unique field again → assert 409 Conflict.
4. POST missing required field → 400 with error code.
5. POST wrong type → 422.
6. POST without auth → 401; with wrong role → 403.
7. POST oversized payload → 413.
8. If accessible, verify DB row exists.

Q. When do you use a mock server?

A. When the real API isn't ready, is rate-limited, costs money, or returns unstable data. Tools: Postman mock servers, WireMock, MockServer.

Example: Our partner's KYC API allows 100 calls per day in their dev env. I built a WireMock with the real responses captured, and use it for most automation — falling back to the real one only for nightly contract tests.

11. SQL & Database Testing

Q. Different types of joins — show with SQL.

A.

```
-- INNER JOIN: only matching rows from both tables
SELECT u.name, o.amount
FROM users u
INNER JOIN orders o ON o.user_id = u.id;

-- LEFT JOIN: all users, with their orders (or null)
SELECT u.name, o.amount
FROM users u
LEFT JOIN orders o ON o.user_id = u.id;

-- RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN, SELF JOIN follow the same pattern
```

Q. Find duplicate emails.

A.

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Q. Second-highest salary — three ways.

A.

```
-- 1. Subquery
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 2. LIMIT/OFFSET
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- 3. Window function
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rnk
  FROM employees
) t WHERE rnk = 2;
```

Q. Nth highest salary.

A.

```
SELECT salary FROM (  
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rnk  
  FROM employees  
) t WHERE rnk = N;
```

Q. DELETE vs TRUNCATE vs DROP — when do you use each?

A.

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Rollback?	Yes (in txn)	Usually no	No
WHERE supported	Yes	No	No
Removes structure	No	No	Yes
Speed	Slowest	Fast	Fastest

Example: In my test setup I use DELETE WHERE for cleanup of test rows so I don't impact other data. For full table reset between test suites we use TRUNCATE.

Q. Primary vs Unique key.

A. Primary key: not null + unique, only one per table — usually the row identifier. Unique key: unique but allows one NULL (in most DBs), multiple allowed per table.

Q. What is normalisation?

A. Organising data to reduce redundancy: 1NF (atomic columns), 2NF (no partial dependency on a composite key), 3NF (no transitive dependency), BCNF. Often denormalised for read-heavy analytics.

Q. ACID properties.

A. Atomicity (all or nothing), **Consistency** (DB constraints hold), **Isolation** (concurrent txns don't interfere), **Durability** (committed changes survive crash). Critical in financial systems.

Q. What do you validate in DB testing?

A. Schema (columns, types, constraints, defaults), data integrity (FKs, uniqueness), CRUD via stored procs, trigger behaviour, performance of slow queries (EXPLAIN plan), data migration correctness (counts and checksums before vs after), backup/restore.

Example: After a recent migration that moved address into a separate table, I wrote a script that counted rows in old vs new tables, checked sample joins matched original addresses, and compared SHA hashes — caught two rows that lost their flat numbers in the migration.

12. AI in QA — The Modern Hot Topic

Q. How is AI changing QA?

A. Several places I've seen real benefit:

- **Test generation** — AI drafts test cases from requirements/AC.
- **Self-healing locators** — when a locator breaks, the tool finds a near-match.
- **Visual testing** — AI compares screenshots and ignores noise like timestamps.
- **Defect triage** — clustering duplicates, predicting severity.
- **Flaky-test detection** — ML models classify reliability.
- **Risk-based prioritisation** — picks the smallest passing test set for a change.
- **Natural-language test authoring** — KaneAI/TestmuAI, Testim, Mabl let you write in plain English.

Example: I use ChatGPT/Claude to draft boundary-value cases from AC and to suggest XPath alternatives when our devs use ugly auto-generated IDs. I always review the output — it can hallucinate locators or methods.

Q. Have you used any AI tools in QA?

A. Yes — ChatGPT for drafting test cases and explaining unfamiliar code, Claude for code review and Java refactoring, Applitools Eyes for visual regression on our login and checkout, and I've explored KaneAI/TestmuAI for natural-language test authoring. I'm careful to verify everything AI produces — especially selectors and SQL.

Q. Show me a good prompt for QA work.

A.

You are a senior QA engineer. Given these acceptance criteria:

- Email must be valid format
- Password ≥ 8 chars with 1 upper, 1 digit, 1 special
- Lockout after 5 failed attempts in 10 minutes
- Successful login redirects to /dashboard

Produce a test case table with: ID, technique (EP/BVA/Decision/State), input data, steps, expected result, priority. Cover happy path, negative, boundary, security (SQL injection, XSS), and concurrency (5 attempts).

Specificity is everything — vague prompts give generic answers.

Q. Self-healing automation — how does it work and what are the risks?

A. When a locator fails, the framework uses heuristics (attribute similarity, DOM diff, visual matching) to find the most likely intended element and updates the locator. Risk: it can mask real UI changes that *should* fail the test. I'd run with self-healing enabled but monitor what it heals — every auto-healed test gets reviewed.

Q. What are AI limitations in QA?

A. Hallucinations — fabricated methods, selectors, or APIs. Privacy — never paste production PII into a public LLM. Explainability — hard to know why the model flagged something. Bias from training data. Vendor lock-in. Treat AI output as a first draft, not a finished test.

Q. Visual testing — how does it work?

A. Capture baseline screenshots, on later runs capture new ones and compute an image diff. Modern tools (Applitools, Percy) use AI to ignore anti-aliasing and dynamic content like timestamps. Catches CSS regressions that DOM-based tests miss entirely.

Example: A button colour was changed accidentally — DOM was identical, all my Selenium asserts passed. Applitools flagged it instantly.

13. Mobile Testing — Concepts & Scenarios

Q. How is mobile testing different from web testing?

A. Mobile adds several dimensions web doesn't have: device fragmentation (hundreds of Android models, multiple iOS versions), network variability (2G/3G/4G/5G/Wi-Fi/offline), interruptions (calls, SMS, notifications, low battery), gestures (tap, swipe, pinch, long-press), orientation (portrait/landscape), background/foreground transitions, OS permissions, and platform-specific behaviour (back button on Android, no back button on iOS).

Example: A bug we missed in desktop QA: app crashed when an incoming call interrupted the upload flow. Web has no equivalent.

Q. Native vs Hybrid vs Web app — and how does testing differ?

A.

- **Native** — built with Swift/Kotlin/Java, installed from store. Best performance and access to device APIs. Tested with Appium/Espresso/XCUITest.
- **Hybrid** — a web app inside a WebView container (Cordova, Ionic, React Native, Flutter — though Flutter compiles native). One codebase, two platforms. Test through Appium with context switching between native and WebView.
- **Mobile web** — runs in the device browser. Test with Selenium/Playwright on mobile browsers.

Q. What mobile-specific scenarios do you test?

A.

- **Interrupts** — incoming call, SMS, alarm, push notification mid-flow.
- **Network** — Wi-Fi to mobile, offline, slow 3G, airplane mode, network toggling.
- **Battery** — low battery, battery saver mode, app behaviour while charging.
- **Orientation** — rotate during a form, screen state preserved.
- **Background** — minimise during checkout, kill from recents, resume after hours.
- **Permissions** — denied, granted later, revoked from settings.
- **Memory/storage** — low storage, low memory, multi-app switching.
- **Install / upgrade / uninstall** — fresh install, upgrade from older version (data migration), uninstall and reinstall.
- **Device fragmentation** — different screen sizes, densities, OS versions, manufacturer customisations (MIUI, OneUI, ColorOS).
- **Gestures** — swipe, pinch, long-press, drag.
- **App store** — store listing rendering, deep links, universal links.
- **Localisation** — RTL languages, currency, date format.

Example: Last release I caught a bug where upgrading from v2.3 to v2.4 wiped users' saved addresses because of a schema change — only reproducible with the upgrade scenario, not fresh install.

Q. Real device vs emulator vs simulator.

A.

	Real device	Emulator (Android)	Simulator (iOS)
Speed	Slowest	Slow	Fast
Hardware sensors	Yes	Mocked	Mocked
Network realism	Real	Mocked	Mocked
Cost / availability	High	Low	Low (Mac required)
Use for	Final validation, perf, real network bugs	Functional dev/test	Functional dev/test

Emulators run x86 versions of Android. Simulators (iOS) don't emulate hardware at all — they run iOS code natively on macOS — so battery, real network, and camera behaviour are not realistic.

Q. How do you decide your device coverage matrix?

A. I look at our analytics — top 10 devices and top 3 OS versions that cover ~80% of users. Add a couple of low-end devices for performance, and one outlier (e.g., a tablet). Cloud labs like BrowserStack/LambdaTest/Sauce fill the long tail.

Example: For our app the matrix was: Samsung S22, OnePlus 9, Pixel 7, Redmi Note 11, iPhone 13, iPhone 14, iPhone SE 2 — covering ~85% of MAU. We dropped support for Android 7 once it dropped below 0.5%.

Q. What is monkey testing?

A. Random input testing — sending random taps, swipes, keystrokes to the app. Android has the built-in `adb shell monkey` tool. Good for stress-testing crashes and ANRs.

Example: Running `adb shell monkey -p com.myapp -v 5000` caught a crash on a debug log statement that fired only when a specific permission dialog was dismissed twice in a row.

Q. What is ANR? How is it different from a crash?

A. ANR = Application Not Responding. Android shows it when the main thread is blocked for > 5 seconds (or 10s broadcast receiver). The app is still alive but unresponsive. A crash is an unhandled exception that terminates the process. Both go in your bug report with logs.

Q. How do you check logs on Android and iOS?

A. Android: `adb logcat` — filter by package or tag, save to a file for the bug report. iOS: device logs via Console.app on macOS, or Xcode's Devices & Simulators window.

Q. How do you test push notifications?

A. Trigger from the actual backend (FCM/APNs) for end-to-end, plus a tool like Firebase Console or curl to APNs for ad-hoc. Tests: foreground vs background vs killed-app behaviour, deep link inside notification, do-not-disturb, batching, custom sound/icon, lock-screen visibility, action buttons.

Q. How would you test an app's offline behaviour?

A. Turn off network, perform actions, then come back online and verify sync. Common bugs: pending writes lost, stale cache served, infinite spinners, retry storms when connection returns. Use BrowserStack/Charles Proxy to simulate slow or flaky networks.

Example: For our note-taking app, I created a note offline, killed the app, reopened — note must persist locally. Then turned Wi-Fi on — note must sync. Then I changed the same note on web — last-write-wins conflict handling was the actual bug we found.

14. Appium — Mobile Automation

Q. What is Appium?

A. An open-source automation tool for native, hybrid, and mobile-web apps on iOS, Android, and Windows. It exposes a WebDriver-compatible API, so you can reuse Selenium skills. One framework, two platforms — Java/Python/JS code that runs on both iOS and Android by changing capabilities.

Q. Appium architecture.

A. Client (your test code) talks to Appium server over HTTP using the W3C protocol. The server forwards commands to platform-specific drivers: UiAutomator2 for Android, XCUITest for iOS. Those drivers talk to the actual device. Appium 2.x splits drivers and plugins so you install only what you need.

Example: For an Android test, my Java code calls

`driver.findElement(AppiumBy.accessibilityId("login"))` → Appium server → UiAutomator2 → device → element → click.

Q. Key desired capabilities you set.

A.

```
// Android
UiAutomator2Options opts = new UiAutomator2Options()
    .setPlatformName("Android")
    .setPlatformVersion("14")
    .setDeviceName("Pixel_7")
    .setApp("/path/to/app-release.apk")
    .setAppPackage("com.myapp")
    .setAppActivity(".MainActivity")
    .setAutomationName("UiAutomator2")
    .setNoReset(false);

AppiumDriver driver = new AndroidDriver(new URL("http://127.0.0.1:4723"), opts);
```

For iOS, swap to XCUITestOptions with `setBundleId` and `setUdid` for real devices.

Q. Locator strategies in Appium.

A.

- **Accessibility ID** — preferred. Cross-platform. Set by devs (content-description on Android, `accessibilityIdentifier` on iOS).
- **ID** — Android resource id like `com.myapp:id/login_btn`.
- **XPath** — works but slow; last resort.
- **iOS Predicate String** — fast: `name == 'Login' AND visible == 1`.
- **iOS Class Chain** — fast: `*/XCUIElementTypeButton[`name == 'Login']`.

- **UIAutomator** on Android: `new UiSelector().text("Login").`
- **Image-based** — using a reference image; useful for canvas-rendered UIs.

Q. How do you inspect mobile elements?

A. Appium Inspector — official GUI that connects to a running session. For Android only I sometimes use UIAutomator Viewer or Android Studio's Layout Inspector. For iOS, Xcode's Accessibility Inspector.

Q. Basic Appium test in Java.

A.

```
UiAutomator2Options opts = new UiAutomator2Options()
    .setPlatformName("Android")
    .setDeviceName("Pixel_7")
    .setApp("/Users/me/app.apk");

AndroidDriver driver = new AndroidDriver(
    new URL("http://127.0.0.1:4723"), opts);

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
wait.until(ExpectedConditions.elementToBeClickable(
    AppiumBy.accessibilityId("emailInput"))).sendKeys("a@b.com");
driver.findElement(AppiumBy.accessibilityId("passwordInput")).sendKeys("pwd");
driver.findElement(AppiumBy.accessibilityId("loginBtn")).click();

driver.quit();
```

Q. How do you perform gestures in Appium?

A. Use W3C Actions (preferred in Appium 2). For Android-specific actions, mobile commands.

```
// Swipe up
Dimension size = driver.manage().window().getSize();
int startX = size.width / 2, startY = (int)(size.height * 0.8);
int endY = (int)(size.height * 0.2);
PointerInput finger = new PointerInput(PointerInput.Kind.TOUCH, "finger");
Sequence swipe = new Sequence(finger, 1)
    .addAction(finger.createPointerMove(Duration.ZERO,
        PointerInput.Origin.viewport(), startX, startY))
    .addAction(finger.createPointerDown(PointerInput.MouseButton.LEFT.asArg()))
    .addAction(finger.createPointerMove(Duration.ofMillis(500),
        PointerInput.Origin.viewport(), startX, endY))
    .addAction(finger.createPointerUp(PointerInput.MouseButton.LEFT.asArg()));
driver.perform(List.of(swipe));
```

Q. Hybrid app — how do you switch context to WebView?

A.

```
Set<String> contexts = driver.getContextHandles();
// contexts contains "NATIVE_APP" and one or more "WEBVIEW_..."
driver.context("WEBVIEW_com.myapp");
// now you can use CSS selectors and HTML locators
driver.context("NATIVE_APP"); // switch back
```

Make sure WebView debugging is enabled on the app side
(WebView.setWebContentsDebuggingEnabled(true)).

Q. How do you start the Appium server?

A. CLI: appium (port 4723 by default). Or programmatically with AppiumServiceBuilder so the framework starts/stops the server with each test run. In CI I let the framework manage it.

Q. noReset vs fullReset?

A. noReset=true — keeps app state (cache, login) between sessions. Faster but flaky if previous state pollutes the test. fullReset=true — uninstalls and reinstalls the app every session. Slow but clean. Default behaviour is in between: app data is cleared but app is not reinstalled.

Q. Common Appium challenges and how you've handled them.

A.

- **Slow execution** — minimised waits, ran in parallel on multiple devices via Appium Grid / Selenium Grid 4.
- **Element not found / stale** — used accessibility ids over XPath; added explicit waits.
- **Device disconnections** — added retry on session creation; pinned a known-good Appium driver version.
- **Permissions popups** — auto-grant with autoGrantPermissions=true capability.
- **Keyboard hiding inputs** — driver.hideKeyboard().
- **Different behaviour on iOS vs Android** — abstracted into a PlatformActions layer in POM so tests stay platform-agnostic.

Example: Our biggest pain was Android emulators dying after 30 minutes. I switched to BrowserStack real devices for nightly, kept emulators for local dev only — solved both flakiness and the cost question.

Q. How would you structure an Appium framework?

A. Same shape as Selenium POM, with a platform abstraction. BasePage takes a generic AppiumDriver. Have separate AndroidLoginPage and IOSLoginPage classes that extend a common interface, instantiated by a factory based on the platform capability. The tests reference only the interface.

15. Load & Performance Testing — JMeter & JUnit

Q. What types of performance testing have you done?

A.

Type	What it answers
Performance test	Is it fast enough under normal load?
Load test	Does it hold up at expected peak?
Stress test	Where does it break?
Spike test	How does it react to sudden surges?
Endurance / Soak	Does it degrade over hours (memory leak)?
Scalability test	Do extra resources actually improve throughput?

Example: Last Diwali peak prep we ran: a load test at 2x last year's peak, a stress test to find breaking point (we hit it at 4500 RPS), and a 12-hour soak. Found a slow memory leak in the cart service.

Q. What is JMeter?

A. Apache JMeter is an open-source Java tool for load and performance testing. It can test HTTP/REST, JDBC, JMS, FTP, SOAP, GraphQL, and more. Used both as a GUI for designing and a CLI for running headless in CI.

Q. JMeter components.

A.

- **Test Plan** — the top-level container.
- **Thread Group** — defines virtual users, ramp-up, loop count, duration.
- **Samplers** — the actual requests (HTTP, JDBC, JUnit, etc.).
- **Logic Controllers** — flow control (loop, if, while, transaction).
- **Config Elements** — defaults like HTTP host, headers, CSV data.
- **Pre/Post Processors** — auth tokens before, response extractors after.
- **Assertions** — pass/fail conditions on response.
- **Timers** — think time between requests (constant, gaussian, etc.).
- **Listeners** — collect and display results (View Results Tree, Aggregate Report, Summary Report). In CI, write results to JTL and post-process.

Q. How do you record a JMeter script?

A. Add an HTTP(S) Test Script Recorder to the Workbench/Test Plan. Set the browser's proxy to JMeter's port (default 8888). Install JMeter's CA cert in the browser for HTTPS. Browse the app —

JMeter captures requests under a Recording Controller. Clean up static asset requests (CSS, images), add correlations for dynamic values (tokens, IDs), parameterise inputs from CSV.

Q. What does ramp-up mean?

A. The time JMeter takes to start all threads. Ramp-up 60s with 100 users = 1 new user per 0.6s. Too short and you get an artificial spike; too long and you don't get sustained load. For a 30-min load test I typically use a 5-min ramp-up, 20-min steady, 5-min ramp-down.

Q. Correlation in JMeter — what and why?

A. Many requests need values returned from previous responses — session tokens, CSRF, dynamic IDs. Correlation captures those values from the first response (Regex / JSON Extractor / CSS Extractor / Boundary Extractor) into a variable, then re-uses them in later requests. Without correlation, login-then-action flows fail.

Example:

```
// JSON Extractor on /login response:
$.token → ${AUTH_TOKEN}

// HTTP Header Manager on subsequent requests:
Authorization: Bearer ${AUTH_TOKEN}
```

Q. Key KPIs in a load test.

A.

- **Throughput** (requests/sec)
- **Response time** — average, median, 90th/95th/99th percentile (percentiles matter more than average)
- **Error rate** (% failed)
- **Concurrent users**
- **CPU / memory / IO** on the server (from monitoring)
- **DB connection-pool usage**

Example: For our search API the SLA was 95th percentile under 500 ms at 500 concurrent users. We measured both the response time and CPU on the search-service pods together.

Q. How do you analyse a bottleneck?

A. Look at where time goes — client-side (test infra), network, app server, DB. Watch APM (NewRelic/Datadog/Grafana) during the test. If CPU on app is at 100%, the bottleneck is the app. If DB CPU is high or query latency rises with users, it's the DB. If neither is busy but response time is high, suspect locks, GC pauses, or external dependencies.

Example: A spike test showed 5xx errors at 1000 RPS. CPU was fine. DB connection pool was at max (100). We bumped to 200 and added connection-pool monitoring — solved.

Q. How do you run JMeter headless in CI?

A.

```
jmeter -n -t TestPlan.jmx -l results.jtl -e -o reportDir \
-Jusers=500 -Jrampup=300 -Jduration=1800
```

-n non-GUI · -t test plan · -l JTL results · -e -o generate HTML report · -J properties from CLI override.

Q. Distributed load testing in JMeter.

A. When one machine can't generate enough load, run JMeter in master-slave mode: master coordinates, slaves generate the actual traffic. Cloud alternative is BlazeMeter or AWS-hosted load generators.

Q. JMeter and JUnit — how do they connect?

A. JMeter has a **JUnit Request Sampler** — it lets you execute existing JUnit test classes as load tests inside a thread group. Useful when you have well-written API tests in JUnit and want to reuse them for load. Steps:

1. Build your JUnit test JAR (compile, no external runner needed).
2. Drop the JAR into JMeter's lib/junit/ folder.
3. Add a JUnit Request sampler in a Thread Group, pick the test class & method.
4. Run with multiple threads — each thread executes the JUnit test.

This is different from JMeter's HTTP Sampler — here JMeter just runs your Java code N times in parallel.

Example:

```
// Plain JUnit test reused inside JMeter for load:
public class UserApiLoadTest {
    @Test
    public void createUser() {
        Response r = RestAssured.given()
            .header("Authorization", "Bearer " + System.getProperty("TOKEN"))
            .body(Map.of("email", UUID.randomUUID() + "@x.com"))
            .contentType(ContentType.JSON)
            .post("https://api.example.com/users");
        assertEquals(201, r.statusCode());
    }
}
```

Build the JAR, put in JMeter lib/junit/, point the JUnit Sampler at createUser.

Q. Other load-testing tools you've heard of?

A.

- **Gatling** — Scala/Java DSL, great reports, lower memory than JMeter.

- **k6** — JavaScript-based, developer-friendly, CI-native.
- **Locust** — Python-based, distributed by default.
- **BlazeMeter** — cloud-hosted JMeter with reporting.
- **NeoLoad** — enterprise, paid.

Q. Difference between performance testing and performance engineering.

A. Performance testing is what we do — measure under load. Performance engineering is broader — designing for performance from the start: capacity planning, architecture choices, profiling, caching strategy, DB indexing. Performance engineers are involved before any test is run.

16. Playwright — Basic Interview Questions

Q. What is Playwright?

A. An open-source browser automation framework from Microsoft. Supports Chromium, Firefox, and WebKit (Safari engine) with a single API in JavaScript/TypeScript, Python, Java, and .NET. Designed for modern web apps — built-in auto-waiting, network interception, parallel execution, and visual debugging.

Q. Playwright vs Selenium — what's the difference?

A.

	Playwright	Selenium
Protocol	Direct CDP / DevTools — single process	WebDriver — separate process per browser
Auto-waiting	Built-in for every action	You write explicit waits
Browser support	Chromium, Firefox, WebKit	All major browsers + many drivers
Mobile	Emulated mobile only	Real mobile via Appium
Network mock	First-class API	Needs CDP or proxy
Speed	Generally faster, less flaky	Stable, mature ecosystem
Multi-tab/window	Native browser-context model	Manual handle switching
Maturity	Younger (2020+)	Mature (since 2004)

Playwright is gaining fast — many product companies are using it for new projects; Selenium is still dominant in enterprises with existing investment.

Q. What is auto-waiting in Playwright?

A. Before every action, Playwright automatically waits for the element to be attached, visible, stable, enabled, and able to receive events. So `page.click()` waits implicitly — no explicit wait code needed for normal cases. It removes 80% of the "wait" boilerplate I write in Selenium.

Example:

```
// Selenium
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(By.id("login")))
    .click();
```



```
// Playwright (Java)
page.click("#login"); // auto-waits
```

Q. What are Playwright's recommended locators?

A. User-facing, role-based locators in this priority:

- `page.getByRole('button', { name: 'Sign in' })`
- `page.getByLabel('Email')`
- `page.getByPlaceholder('Search...')`
- `page.getByText('Welcome')`
- `page.getByTestId('login-btn')` — when devs add `data-testid`
- CSS / XPath only as a last resort

Role-based locators are accessibility-aligned and survive UI refactors better than CSS selectors.

Q. How does Playwright handle multiple tabs and windows?

A. Through *browser contexts* and *pages*. A context is like an isolated browser profile; a page is a tab. Listening for the page event on a context catches new tabs.

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('a[target=_blank]')
]);
await newPage.waitForLoadState();
```

Much cleaner than Selenium's window-handle iteration.

Q. What is Playwright's trace viewer?

A. Playwright records a trace (DOM snapshots, screenshots, network log, console log, action timeline) and lets you replay it visually after the test. For a failed CI test you download `trace.zip`, open it in `npx playwright show-trace`, and step through what happened frame by frame — by far the best debugging experience I've used.

Q. What is Playwright codegen?

A. A CLI command (`npx playwright codegen https://example.com`) that opens the site, records your actions, and emits Playwright code with role-based locators. Useful for quick scaffolding — not a replacement for handwritten tests.

Q. How do you intercept and mock network requests?

A.

```
await page.route('**/api/users', route => {
  route.fulfill({ status: 200, body: JSON.stringify([
    { id: 1, name: 'Test' }
  ]) });
});
```

You can mock APIs the page calls, abort certain requests (analytics, ads) to speed tests, or modify responses to test edge cases — without changing the backend.

Q. Can Playwright do API testing too?

A. Yes. Playwright has a built-in API testing API (request fixture) that uses the same context as your browser — so cookies and auth flow naturally between API and UI calls.

```
const response = await request.post('/api/login', {
  data: { email: 'a@b.com', password: 'pwd' }
});
expect(response.status()).toBe(200);
```

Q. How does parallel execution work in Playwright?

A. By default Playwright runs test files in parallel across multiple workers (separate processes). Inside a file, tests run serially unless you enable parallel-within-file. You can also *shard* the test run across machines in CI: `npx playwright test --shard=1/4`.

Q. Page Object Model in Playwright.

A. Same pattern as Selenium — class per page with locators and methods. Cleaner because of the built-in waits.

```
class LoginPage {
  constructor(page) { this.page = page; }
  emailInput() { return this.page.getByLabel('Email'); }
  passwordInput() { return this.page.getByLabel('Password'); }
  loginButton() { return this.page.getByRole('button', { name: 'Sign in' }); }

  async loginAs(email, pwd) {
    await this.emailInput().fill(email);
    await this.passwordInput().fill(pwd);
    await this.loginButton().click();
  }
}
```

Q. How do you handle authentication so you don't log in for every test?

A. Use **storage state**. Run a one-time login script, save cookies/localStorage to a JSON file, then load it for every test:

```
// global-setup.ts
const context = await browser.newContext();
await context.storageState({ path: 'auth.json' });

// playwright.config.ts
use: { storageState: 'auth.json' }
```

Saves a significant chunk of test time on suites with hundreds of tests.

Q. When would you choose Playwright over Selenium for a new project?

A. For a modern SPA-heavy web app where speed and stability matter and we don't need real-device mobile coverage. Selenium still wins for: enterprise legacy projects with existing investment, real-device mobile testing (via Appium), and unusual browser combinations. For most new web automation work in 2026, I'd default to Playwright.

17. Git & GitHub for QA

Q. What is Git? How is it different from GitHub?

A. Git is a distributed version-control system that lives on your machine and tracks every change to your code. GitHub is a cloud hosting platform for Git repositories with collaboration features on top — pull requests, code reviews, issues, Actions for CI. Alternatives to GitHub: GitLab, Bitbucket, Azure Repos.

Example: In my team we use Git locally for branching and commits, GitHub for pushing branches and raising PRs that go through review before merging to main.

Q. Daily Git commands you use.

A.

```
git clone <url>                # first time
git status                     # what's changed
git pull --rebase              # update local with remote
git checkout -b feat/login-tests # new branch
git add src/test/...          # stage
git commit -m "add login regression tests"
git push -u origin feat/login-tests
git log --oneline --graph -10  # history
git diff HEAD~1                # what changed in the last commit
git branch                     # list branches
git checkout main              # switch branches
git merge feat/login-tests     # bring branch in
git stash                      # park uncommitted changes
git stash pop                  # bring them back
```

Q. Branching strategies — what have you used?

A.

- **Git Flow** — long-lived main, develop, plus feature/*, release/*, hotfix/* branches. Suits release-train products.
- **GitHub Flow** — only main + short-lived feature branches. PRs reviewed and merged frequently. Simpler, suits continuous deployment.
- **Trunk-based development** — almost everything goes straight to main behind feature flags. Used by very mature teams with strong test automation.

Example: My current team uses GitHub Flow. I branch feat/<jira-id>-short-name, commit, push, raise a PR, get it reviewed, and merge — usually within a day.

Q. Merge vs Rebase — when do you use each?

A. Merge brings two branches together and preserves the full history with a merge commit. **Rebase** replays your commits on top of another branch, producing a linear history. My rule: *merge for shared branches, rebase for local cleanup before pushing*. Never rebase a branch other people have based work on — it rewrites history they depend on.

Example:

```
# Update my feature branch with the latest main, keep history linear:
git checkout feat/login-tests
git fetch origin
git rebase origin/main          # OR: git merge origin/main
# resolve conflicts if any
git push --force-with-lease     # only after rebase
```

Q. How do you resolve a merge conflict?

A. Git marks conflicting sections in the file with <<<<<<, =====, and >>>>>>. Steps: open each conflicted file, decide what to keep (mine, theirs, or merged), remove the markers, save, `git add` the file, then `git commit` (or `git rebase --continue` if you were rebasing). If overwhelmed, `git merge --abort` or `git rebase --abort` backs out cleanly.

Example: Last week a teammate and I edited the same locator in `LoginPage.java`. I opened the file in IntelliJ, used the three-way merge tool to keep both my new method and her renamed field, ran the test locally to confirm, then committed.

Q. git reset vs revert vs checkout.

A.

Command	What it does	Safe on shared branches?
<code>git reset --soft <commit></code>	Moves HEAD back, keeps changes staged	No (rewrites history)
<code>git reset --mixed</code> (default)	Moves HEAD, unstages	No
<code>git reset --hard</code>	Moves HEAD, wipes changes	No — destructive
<code>git revert <commit></code>	Creates a NEW commit that undoes another	Yes — safe
<code>git checkout <branch></code>	Switches branch	Yes
<code>git checkout -- <file></code>	Discards local changes to that file	Local only

On a shared branch always use `revert`. `reset --hard` is for personal branches only.

Q. Walk me through your PR workflow.

A.

1. Create branch feat/JIRA-1234-add-checkout-tests off latest main.
2. Commit in small logical chunks with clear messages.
3. Push and open a PR with a description linking the JIRA ticket.
4. CI runs automatically (compile, unit tests, smoke automation).
5. Request review from one dev + my QA lead.
6. Address comments — small fixes as new commits; if the change is big, force-push after rebase.
7. Once approved and CI green, squash-merge into main and delete the branch.

Q. How do you undo a bad commit that's already been pushed?

A. If the branch is shared, use `git revert <commit-sha>` — it creates a new commit that undoes the bad one without rewriting history. If the branch is personal, you can reset and force-push (`git push -force-with-lease`). Never force-push to main.

Q. git stash — when do you use it?

A. When I have uncommitted work and need to switch branches quickly to handle something else (a P1 hotfix, an urgent review). `git stash` parks the changes; `git stash pop` brings them back. `git stash list` shows what's parked.

Q. What goes in .gitignore for a QA automation project?

A.

```
target/           # Maven build output
.idea/ *.iml      # IntelliJ
.vscode/          # VS Code
test-output/      # TestNG reports
allure-results/   # Allure
extent-report/    # Extent
screenshots/      # captured on failure
*.log
.env              # credentials — never commit
config-local.properties
*.class
```

Q. Tags vs branches.

A. A **branch** is a movable pointer that advances with each new commit. A **tag** is an immutable label pointing at a specific commit — typically used to mark releases like v2.4.0. Lightweight tags are just pointers; annotated tags carry a message and metadata (`git tag -a v2.4.0 -m "Diwali release"`).

Q. git cherry-pick — what is it?

A. Apply a specific commit from one branch onto another. Useful when a single fix on a feature branch needs to also go to a release branch. `git cherry-pick <sha>` — but use it sparingly; if you cherry-pick many commits you can get yourself into divergent histories.

Q. What is HEAD? What is "detached HEAD"?

A. HEAD is a symbolic pointer to your current location — usually a branch tip. Detached HEAD means HEAD is pointing directly at a commit, not a branch — common when you check out a tag or specific commit. Any new commits there are orphaned unless you create a branch first.

18. CI/CD — Jenkins & GitHub Actions

Q. What is CI/CD?

A. **CI (Continuous Integration)**: developers merge code to a shared branch frequently; every merge triggers an automated build and test run, so integration issues surface in minutes not weeks. **CD (Continuous Delivery)**: the build that passes CI is automatically packaged and deployable to any environment with one click. **Continuous Deployment** goes one step further — the passing build deploys to production automatically.

Q. Why should QA care about CI/CD?

A. Because QA isn't a checkpoint at the end any more — our automation lives inside the pipeline. On every PR a smoke suite runs; on merge to main a fuller regression runs; nightly we run perf and visual tests. We move from "give us a build to test" to "the build proves itself by passing our tests automatically."

Example: In our team a PR can't merge unless the smoke suite (40 tests, 6 minutes) is green. That single rule reduced "broken-on-main" incidents from weekly to almost zero.

Q. Jenkins basics — explain controller (master), agent (slave), job, pipeline.

A. **Controller** orchestrates — schedules jobs, serves the UI. **Agents** are worker machines that actually run the builds (so the controller stays responsive). A **job** is one unit of work — can be freestyle (UI-configured) or pipeline (code in a Jenkinsfile). A **pipeline** is a sequence of stages defined as code.

Q. Freestyle vs Pipeline job — which do you prefer?

A. Pipeline, because the Jenkinsfile lives in the repo alongside the code — version-controlled, reviewable, and reproducible. Freestyle jobs are configured by clicking in the UI and are hard to audit or migrate. The modern Jenkins answer is "Pipeline (Declarative)."

Q. Declarative vs Scripted Pipeline.

A. **Declarative** uses the structured pipeline { agent {} stages { stage {} } } syntax — readable, predictable, easy to lint. **Scripted** is Groovy in a node { ... } block — more flexible but harder to maintain. Use Declarative; drop into a script {} block for the rare bits that need Groovy.

Q. Sample Jenkinsfile for a Selenium-Maven project.

A.

```
pipeline {
  agent any
  tools { maven 'Maven-3.9'; jdk 'JDK-17' }
  parameters {
    choice(name: 'BROWSER', choices: ['chrome','edge','firefox'], description:
'Browser')
    choice(name: 'SUITE', choices: ['smoke','regression'], description:
```



```

'Suite')
}
options { timeout(time: 30, unit: 'MINUTES');
buildDiscarder(logRotator(numToKeepStr: '20')) }
stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build')     { steps { sh 'mvn -B clean compile' } }
    stage('Test') {
        steps {
            sh "mvn -B test -Dbrowser=${params.BROWSER} -
Psurefire-${params.SUITE} -Dheadless=true"
        }
    }
}
post {
    always {
        junit 'target/surefire-reports/*.xml'
        publishHTML([reportDir: 'test-output/extent', reportFiles: '*.html',
            reportName: 'Extent Report', keepAll: true])
        archiveArtifacts artifacts: 'test-output/**/*', allowEmptyArchive: true
    }
    failure {
        slackSend channel: '#qa-alerts', color: 'danger',
            message: "Build ${env.JOB_NAME} #${env.BUILD_NUMBER} failed"
    }
}
}

```

Q. How do you publish test reports in Jenkins?

A. JUnit XML via the built-in junit step (works for TestNG output too). HTML reports (Extent, Cucumber) via the HTML Publisher plugin. For richer dashboards, the Allure plugin reads allure-results/ and renders a full report. Pictures of failures go in via screenshot files attached to those reports.

Q. How do you parameterise a Jenkins job?

A. Declarative pipelines have a parameters { } block — string, choice, boolean, password, file. Users select values when triggering the build. In Jenkinsfile I refer to them with \${params.NAME}.

Q. How do you schedule a Jenkins job?

A. In Build Triggers, "Build periodically" with cron syntax. Example to run nightly at 1 AM: 0 1 * * *. For Pipeline, the same can be expressed inside the Jenkinsfile via the triggers { cron('H 1 * * *') } block. The H spreads load randomly.

Q. How do you send Slack/email notifications on failure?

A. In the post section, use failure { ... }. For Slack, install the Slack Notification plugin and call slackSend. For email, the Email Extension plugin and emailext. Always include build URL, branch, and a one-line summary so the on-call doesn't have to dig.

Q. How do you run cross-browser tests in parallel in Jenkins?

A. A `parallel` block inside a stage:

```
stage('Cross-browser') {
  parallel {
    stage('Chrome') { steps { sh 'mvn test -Dbrowser=chrome' } }
    stage('Edge')   { steps { sh 'mvn test -Dbrowser=edge' } }
    stage('Firefox') { steps { sh 'mvn test -Dbrowser=firefox' } }
  }
}
```

For real scale we point Selenium at a Grid (or BrowserStack) so each browser runs on a fresh node.

Q. What is GitHub Actions?

A. GitHub's built-in CI/CD. Workflows are YAML files in `.github/workflows/`. No separate server to maintain — runs on GitHub-hosted runners or self-hosted ones. A workflow has events that trigger it, jobs that group steps, and steps that run commands or pre-built actions from the marketplace.

Q. Sample GitHub Actions workflow for a Selenium-Maven project.

A.

```
name: QA Automation
on:
  push:
    branches: [main]
  pull_request:
  schedule:
    - cron: '0 1 * * *' # nightly regression
  workflow_dispatch: # manual

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        browser: [chrome, firefox]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17', cache: maven }
      - name: Run tests
        env:
          TEST_USER: ${ secrets.TEST_USER }
          TEST_PASS: ${ secrets.TEST_PASS }
        run: mvn -B test -Dbrowser=${ matrix.browser } -Dheadless=true
      - uses: actions/upload-artifact@v4
```

```
if: always()
with:
  name: extent-report-${{ matrix.browser }}
  path: test-output/extent/
```

Q. How do you handle secrets/credentials in CI?

A. Never in code. In Jenkins, use the Credentials store and the `withCredentials` step. In GitHub Actions, define repository or organisation Secrets and access them as `${{ secrets.NAME }}` — they're masked in logs. For richer needs, integrate Vault, AWS Secrets Manager, or Azure Key Vault.

Q. What is a matrix build?

A. Running the same job across a grid of variants — browser × OS × Java version. GitHub Actions has first-class support via `strategy.matrix`. Saves a lot of duplicate YAML. Use `fail-fast: false` if you want every variant to run even when one fails.

Q. Common CI pitfalls for QA automation.

A.

- Tests assume headed mode — fail in CI's headless runner.
- Hard-coded paths or localhost URLs.
- Driver/browser version mismatches — let Selenium Manager handle drivers.
- No artifacts on failure — collect screenshots, HARs, logs always (`if: always()`).
- Long pipelines because tests run sequentially — parallelise.
- Flaky tests blocking merges — quarantine flaky tests, fix root cause, don't normalise retries.
- Hardcoded credentials. Use secrets every time.
- State leaking between runs — clean up DB / test data; use fresh containers.

Q. CI/CD pipeline stages you'd recommend for a QA-heavy project.

A.

1. **Checkout** — pull the code.
2. **Static analysis** — Checkstyle/SpotBugs/SonarQube.
3. **Compile** — fail fast if it doesn't build.
4. **Unit tests** — quick, broad coverage.
5. **API tests** — Rest Assured suite.
6. **UI smoke** — 5–10 minutes, critical paths.
7. **Deploy to QA env** — for nightly: deploy then run full regression.
8. **Reports** — publish Extent/Allure/JUnit XML, archive artifacts.
9. **Notify** — Slack/email on failure with build link.

19. Maven — Build & Dependency Management

Q. What is Maven and why do we use it?

A. Maven is a Java build automation and dependency management tool. It standardises project layout, manages dependencies (auto-downloading JARs from Maven Central), and runs lifecycle phases (compile, test, package, install). Reproducible builds anywhere with the same `pom.xml` and a Maven install.

Q. Maven vs Gradle.

A.

	Maven	Gradle
Config	XML (<code>pom.xml</code>)	Groovy or Kotlin DSL (<code>build.gradle</code>)
Speed	Slower	Faster (incremental + daemon)
Ecosystem	Mature, ubiquitous	Modern, newer
Learning curve	Gentle	Steeper
Adoption in QA frameworks	Dominant	Growing

Most QA automation tutorials and frameworks I've seen still use Maven, so that's what I default to.

Q. Standard Maven project structure.

A.

my-project/	
├─ pom.xml	
├─ src/	
│ ├─ main/	
│ │ ├─ java/	application code
│ │ └─ resources/	config, properties
│ └─ test/	
│ ├─ java/	test code
│ └─ resources/	test data, <code>testng.xml</code> , features
└─ target/	build output (gitignored)

Maven's "convention over configuration" — follow this structure and you barely have to configure anything.

Q. Key sections in a `pom.xml`.

A.

```

<project>
  <groupId>com.example.qa</groupId>
  <artifactId>automation-framework</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <selenium.version>4.27.0</selenium.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>${selenium.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin> ... surefire / failsafe / compiler ... </plugin>
    </plugins>
  </build>

  <profiles> ... </profiles>
</project>

```

GAV (groupId + artifactId + version) uniquely identifies an artifact in any Maven repository.

Q. Maven build lifecycle — list the phases.

A. The default lifecycle has these phases (a phase runs all phases before it):

1. **validate** — project structure OK
2. **compile** — compile source
3. **test** — run unit tests (Surefire)
4. **package** — bundle compiled output (JAR/WAR)
5. **verify** — run integration tests (Failsafe) and quality checks
6. **install** — copy package to local repo ~/.m2/repository
7. **deploy** — copy to remote repo

There are also two other lifecycles: **clean** (deletes target/) and **site** (generates project documentation).

Q. Difference between `mvn install` and `mvn clean install`.

A. `install` runs `validate` → `compile` → `test` → `package` → `verify` → `install`. `clean install` first deletes the `target/` directory, removing stale class files. Use `clean install` when you've renamed/removed classes, switched branches, or are debugging unexpected build behaviour.

Q. Surefire vs Failsafe plugin.

A.

	Surefire	Failsafe
Runs	Unit tests	Integration tests
Phase	test	integration-test + verify
Default file pattern	*Test.java, *Tests.java, Test*.java	*IT.java, IT*.java
Build fails on test failure?	Immediately	Only at verify

The Failsafe split is important — if integration tests fail, you still want post-test cleanup (stop servers, drop DB) to run before the build fails.

Q. Common dependency scopes.

A.

- **compile** (default) — available everywhere; packaged.
- **provided** — available at compile/test but expected from the container (Servlet API).
- **runtime** — not needed for compile, needed at runtime (JDBC drivers).
- **test** — only on the test classpath (TestNG, Selenium, Rest Assured).
- **system** — like provided but local path (avoid).
- **import** — only inside `<dependencyManagement>` for BOMs.

Q. What is a transitive dependency and how do you debug conflicts?

A. A transitive dependency is one your dependencies pull in. If two of your dependencies pull in different versions of the same library, Maven resolves it by "nearest wins" (shortest path in the tree). To see what's happening:

```
mvn dependency:tree
mvn dependency:tree -Dincludes=com.fasterxml.jackson.core:jackson-databind
```

If you need to force a version, declare it explicitly in your `<dependencies>` — direct declarations win.

Q. Maven profiles — what and why?

A. Profiles let you switch sets of configuration on/off based on env, OS, or CLI flag. I use them for environment-specific URLs, browser choice, or to run different test suites.

```
<profiles>
  <profile>
    <id>qa</id>
    <properties><env.url>https://qa.example.com</env.url></properties>
  </profile>
  <profile>
    <id>regression</id>
    <build><plugins>
      <plugin><artifactId>maven-surefire-plugin</artifactId><configuration>
```

```

        <suiteXmlFiles><suiteXmlFile>regression.xml</suiteXmlFile></suiteXmlFiles>
    </configuration></plugin>
</plugins></build>
</profile>
</profiles>

```

Activate via `mvn test -Pqa -Pregression`.

Q. Properties and parameterisation in Maven.

A. Define properties at the top of `pom.xml` for versions, then reuse with `${prop.name}`. Override from the CLI with `-D`.

```

<properties>
  <selenium.version>4.27.0</selenium.version>
  <browser>chrome</browser>
</properties>
...
<version>${selenium.version}</version>

```

At runtime: `mvn test -Dbrowser=edge -Dselenium.version=4.28.0`.

Q. What is a multi-module project?

A. A parent `pom` with `<modules>` listing child modules. Children inherit settings and can depend on each other. Useful when you have shared utilities + multiple test suites (smoke / API / mobile) in one repo.

```

<modules>
  <module>framework-core</module>
  <module>web-tests</module>
  <module>api-tests</module>
  <module>mobile-tests</module>
</modules>

```

Q. Useful Maven commands you run daily.

A.

```

mvn -v                # version
mvn clean              # delete target/
mvn compile            # compile main sources
mvn test               # run unit tests
mvn -Dtest=LoginTest test  # run one test class
mvn -Dtest=LoginTest#login_valid test  # run one method
mvn -DskipTests package  # build without tests (rare)
mvn install            # build + install to local repo
mvn dependency:tree     # transitive dependency graph
mvn dependency:purge-local-repository  # nuke cache (rare)

```

```
mvn -X test          # debug logs
mvn -B test          # batch (non-interactive, CI)
```


20. Java & OOPs — Core Interview Questions

17.1 OOPs Fundamentals

Q. What is Object-Oriented Programming and what are its four pillars?

A. OOP is a paradigm that models software around *objects* — units of state and behaviour — rather than around functions. The four pillars are **Encapsulation, Inheritance, Polymorphism**, and **Abstraction**. In Java they map directly to language features — classes, extends, overloading/overriding, and abstract classes/interfaces.

Example: In my Selenium framework: page objects encapsulate locators (Encapsulation), every page extends BasePage (Inheritance), the same `click()` method works for any locator type (Polymorphism), and BasePage is abstract so tests must use concrete subclasses (Abstraction).

What they're really asking: A baseline check — they'll dig into each pillar individually after this.

Q. Class vs Object — explain with an example.

A. A class is the blueprint — definition of state and behaviour. An object is a runtime instance of that class — actual data in memory.

Example:

```
class Car {  
    String model;  
    int year;  
    void start() { System.out.println(model + " starting"); }  
}  
  
Car c1 = new Car();    // c1 is an OBJECT of class Car  
c1.model = "Swift";  
c1.start();
```

You can have one class and many objects — same blueprint, different state.

Q. Encapsulation — what and why?

A. Encapsulation = binding data and behaviour together inside a class, and hiding the data with access modifiers. We expose state through controlled getter/setter methods. Benefits: validation in setters, ability to change internal representation without breaking callers, and a clear API.

Example:

```
class Account {  
    private double balance;    // hidden
```

```

    public double getBalance() { return balance; }
    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Amount must be
positive");
        this.balance += amt;
    }
}

```

Without encapsulation, any caller could do `account.balance = -100`. With it, we enforce business rules at the boundary.

Q. What is Inheritance? List the types Java supports.

A. Inheritance lets a class acquire fields and methods of another class via `extends`. Java supports:

- **Single** — A extends B
- **Multilevel** — A extends B extends C
- **Hierarchical** — B and C both extend A
- **Multiple inheritance of type** via interfaces — a class can implement many interfaces

Java does **not** support multiple inheritance of *classes*, to avoid the *diamond problem* — two parents defining the same method with different bodies.

Example:

```

class Vehicle { void move() {} }
class Car extends Vehicle { void honk() {} }           // single
class SportsCar extends Car {}                         // multilevel

interface Drivable { void drive(); }
interface Maintainable { void service(); }
class Truck extends Vehicle implements Drivable, Maintainable {} // multiple
inheritance of type

```

Q. IS-A vs HAS-A relationship.

A. **IS-A** = inheritance ("a Car IS-A Vehicle") — use `extends` or `implements`. **HAS-A** = composition ("a Car HAS-A Engine") — one class holds a reference to another. Composition is generally preferred over inheritance because it's looser coupling and easier to change.

Example:

```

class Engine {}
class Car extends Vehicle {           // IS-A
    private Engine engine;           // HAS-A
}

```

Q. What is Polymorphism? Give both kinds with examples.

A. Polymorphism = "many forms" — same name, different behaviour. Two kinds:

- **Compile-time (static / overloading)** — same method name, different parameters, same class. Bound at compile time.
- **Runtime (dynamic / overriding)** — subclass redefines a parent method with the same signature. Bound at runtime via the actual object type.

Example:

```
// Overloading (compile-time)
class Calculator {
    int add(int a, int b)          { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c)   { return a + b + c; }
}

// Overriding (runtime)
class Animal { void sound() { System.out.println("generic"); } }
class Dog extends Animal { @Override void sound() { System.out.println("bark"); } }

Animal a = new Dog();
a.sound();    // prints "bark" — decided at runtime
```

Q. Method overloading vs overriding — give me the rules.

A.

	Overloading	Overriding
Where	Same class (or subclass)	Subclass
Signature	Must differ in parameters	Must be identical
Return type	Can differ	Same or covariant (subtype)
Access modifier	Can be anything	Cannot be more restrictive
Exceptions	Can differ	Cannot throw new broader checked exceptions
Binding	Compile-time (static)	Runtime (dynamic)
Static methods	Can be overloaded	Cannot be overridden (only hidden)
final / private methods	Can be overloaded	Cannot be overridden

Q. Can we override a static method?

A. No — you can *hide* it but not override it. If a subclass declares a static method with the same signature, the call binds to the declared type, not the runtime type. Overriding is about dynamic

dispatch; static methods don't participate.

Example:

```
class A { static void hello() { System.out.println("A"); } }
class B extends A { static void hello() { System.out.println("B"); } }

A obj = new B();
obj.hello();    // prints "A" – not overridden, just hidden
```

Q. Can we override private methods? Final methods? main()?

A. No, no, and "main can be overloaded but not overridden." Private methods are not visible to subclasses. Final methods are sealed. main is static — so the previous rule applies.

Q. Abstraction — what is it and how is it implemented in Java?

A. Abstraction = exposing only the relevant behaviour, hiding the implementation. In Java, achieved through **abstract classes** (0–100% abstract) and **interfaces** (100% abstract by default before Java 8). The user of an abstract type doesn't need to know the concrete class.

Example:

```
abstract class Payment {
    abstract void pay(double amount);    // contract
    void log() { System.out.println("Payment logged"); } // concrete, shared
}

class UpiPayment extends Payment {
    @Override void pay(double a) { /* UPI call */ }
}

class CardPayment extends Payment {
    @Override void pay(double a) { /* card gateway */ }
}

Payment p = new UpiPayment();    // caller doesn't know it's UPI
p.pay(500);
```

Q. Abstract class vs Interface — deep difference.

A.

Abstract class	Interface
Can have instance state, constructors	No instance state; constants only (public static final)
Single inheritance (extends one)	Multiple inheritance of type (implements many)

Mix of abstract and concrete methods	All abstract until Java 8; default/static/private methods added later
Any access modifier on methods	Methods implicitly public
Can have main() and be tested directly	Cannot be instantiated; no constructor
Use when classes share state & partial behaviour	Use to define a capability/contract across unrelated classes

Rule of thumb: **IS-A and share behaviour** → abstract class. **CAN-DO capability** → interface.

Q. What changed in interfaces from Java 8 and Java 9?

A. Java 8 added **default methods** (concrete methods in an interface) and **static methods**. Default methods let you add behaviour to an interface without breaking implementers. Java 9 added **private methods** in interfaces — to share code between default methods.

Example:

```
interface Vehicle {
    void move();
    default void honk() { System.out.println("Beep"); } // Java 8
    static Vehicle defaultVehicle() { return new Bike(); } // Java 8
    private void logSpeed() { /* shared helper */ } // Java 9
}
```

Q. What is the diamond problem and how does Java solve it?

A. If a class could inherit from two classes that define the same method with different bodies, the compiler wouldn't know which to use — that's the diamond. Java avoids it by disallowing multiple inheritance of classes. Interfaces can have default methods, so the diamond exists there too — Java requires the implementing class to override the conflicting method.

Q. What is a marker interface?

A. An interface with no methods — used purely to tag classes with a property the JVM or framework can check. Examples: `Serializable`, `Cloneable`, `Remote`. Modern Java prefers annotations for the same job.

Q. What is a functional interface?

A. An interface with exactly one abstract method. Targets of lambda expressions. Common ones in `java.util.function`: `Function<T,R>`, `Predicate<T>`, `Consumer<T>`, `Supplier<T>`, `BiFunction`. `@FunctionalInterface` annotation makes the compiler enforce the single-abstract-method rule.

Example:

```
@FunctionalInterface
interface Greet { String say(String name); }
```

```
Greet g = name -> "Hello " + name;  
System.out.println(g.say("Akshat"));
```

17.2 Constructors, this, super

Q. What is a constructor? What types are there?

A. A constructor initialises a new object. It has the same name as the class and no return type. Types:

- **Default** — no-arg, auto-generated by the compiler if you don't declare any constructor.
- **Parameterized** — takes arguments.
- **Copy** — Java doesn't provide one built-in; you write one that takes another instance.

Example:

```
class User {  
    String name;  
    int age;  
    User() {} // default  
    User(String n, int a) { name = n; age = a; } // parameterized  
    User(User other) { this(other.name, other.age); } // copy  
}
```

Q. What is constructor chaining? Difference between this() and super()?

A. Constructor chaining is calling one constructor from another — within the same class (`this(...)`) or to the parent class (`super(...)`). It must be the first statement in the constructor. If you don't call either, the compiler inserts `super()` (no-arg) automatically.

Example:

```
class Vehicle {  
    Vehicle(String type) { System.out.println("Vehicle " + type); }  
}  
class Car extends Vehicle {  
    Car() { this("Default"); } // calls Car(String)  
    Car(String m) { super("Car"); System.out.println("Car " + m); } // calls  
    Vehicle(String)  
}
```

Q. Can a constructor be private? Final? Static?

A. Private — yes, used in Singleton pattern. **Final** — no, constructors aren't inherited so it makes no sense. **Static** — no, a constructor instantiates an object; static means "no instance."

Q. Difference between `this` and `super`.

A. `this` refers to the current object — used to disambiguate fields, call another constructor of the same class, or pass the current object. `super` refers to the parent — used to call parent's method or constructor, or access shadowed parent fields.

17.3 Static, Final, Modifiers

Q. What does the `static` keyword do?

A. Belongs to the class, not an instance. You can have static variables (shared across all objects), static methods (called without an instance), static blocks (run once when the class is loaded), and static nested classes.

Example:

```
class Counter {  
    static int count = 0;          // shared  
    static { count = 100; }        // static initialiser block – runs once  
    Counter() { count++; }  
    static int getCount() { return count; }  
}
```

Q. Can a static method access non-static members?

A. No — directly. There's no `this` in a static context. You can access non-static members only via an explicit object reference.

Q. What does `final` mean for variable, method, class?

A.

- **final variable** — can be assigned once. Must be initialised at declaration, in a constructor, or in an instance/static initialiser.
- **final method** — cannot be overridden.
- **final class** — cannot be extended. `String`, `Integer`, `Math` are final.

Q. `final` vs `finally` vs `finalize`.

A. final: keyword (no override / no extend / no reassign). **finally:** try block that always runs. **finalize():** deprecated GC callback — don't use; use try-with-resources or explicit cleanup instead.

Q. Access modifiers — list all four.

A.

Modifier	Same class	Same package	Subclass (any package)	Anywhere
private	✓			
default (no keyword)	✓	✓		

protected	✓	✓	✓	
public	✓	✓	✓	✓

17.4 JVM, JDK, JRE, Memory, GC

Q. JDK vs JRE vs JVM.

A. **JVM** — Java Virtual Machine, runs compiled bytecode. **JRE** — Java Runtime Environment = JVM + core libraries needed to run Java programs. **JDK** — Java Development Kit = JRE + compiler + tools (javac, javadoc, jar, jshell). As a developer I install the JDK; an end user only needs the JRE.

Q. JVM memory areas.

A.

- **Method Area / Metaspace** — class metadata, static variables, constants.
- **Heap** — all objects and instance variables. Shared across threads. GC works here.
- **Stack** — one per thread; stores frames for method calls (local variables, partial results). Cleaned up when the method returns.
- **PC Register** — per-thread pointer to the current instruction.
- **Native Method Stack** — for native (JNI) calls.

Example: When you call `new User()`, the `User` object lives in the heap; the reference variable lives in the stack frame of the method that created it.

Q. What is Garbage Collection? How does it work at a high level?

A. GC is JVM's automatic memory management. The heap is split into generations — Young (Eden + 2 Survivor spaces) and Old. New objects start in Eden; survivors of a few cycles get promoted to Old. Minor GCs clean Young frequently and quickly; Major (Full) GCs clean Old and are slower. Common collectors: G1 (default since Java 9), ZGC, Parallel.

Q. Can you force garbage collection?

A. You can *suggest* with `System.gc()`, but the JVM is free to ignore it. In real code you never rely on it.

Q. What is a memory leak in Java despite GC?

A. An object remains reachable from a GC root (e.g., a static collection, listener, `ThreadLocal`) so GC can't collect it, even though the application no longer needs it. Classic culprits: forgotten listeners, static caches, unclosed connections, `ThreadLocals` in thread pools.

17.5 String, Wrappers, Equality

Q. Why is String immutable in Java?

A.

- **Security** — usernames, file paths, URLs passed across boundaries can't be silently mutated.
- **Thread safety** — immutable objects need no synchronisation.

- **String pool** — multiple references can safely share the same underlying object.
- **hashCode caching** — hashCode is computed once and cached, making String an efficient HashMap key.

Q. What is the String pool? How does it work?

A. A special area in the heap (since Java 7) holding string literals. When you write "hello", the JVM checks the pool — if it's there, the reference is reused; if not, a new String is added. `new String("hello")` bypasses the pool and always creates a fresh object on the heap. `intern()` forces a string to be added to / retrieved from the pool.

Example:

```
String a = "hello";
String b = "hello";
String c = new String("hello");
System.out.println(a == b);           // true  - same pool reference
System.out.println(a == c);           // false - c is a new object
System.out.println(a == c.intern());  // true  - intern returns pool reference
```

Q. String vs StringBuilder vs StringBuffer.

A.

	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread-safe?	Yes (immutable)	No	Yes (synchronised)
Speed	Slow for concat in loops	Fast	Slower than StringBuilder
Use when	Few changes	Single-threaded heavy concat	Multi-threaded concat (rare)

Q. == vs equals() vs hashCode() — explain the contract.

A. `==` compares references for objects, values for primitives. `equals()` compares logical equality.

Contract: if `a.equals(b)` then `a.hashCode() == b.hashCode()` must also hold. Violating it breaks HashMap, HashSet, and Hashtable for your objects — they'd fail to find inserted entries.

Example:

```
class Point {
    int x, y;
    @Override public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
}
```

```
@Override public int hashCode() { return Objects.hash(x, y); }  
}
```

Q. What are Wrapper classes? What is autoboxing / unboxing?

A. Each primitive has a corresponding object class: `int` → `Integer`, `long` → `Long`, `double` → `Double`, etc. Wrappers let primitives participate in collections (which only hold objects) and offer utility methods.

Autoboxing = primitive to wrapper automatically (`Integer x = 5;`). **Unboxing** = the reverse (`int y = x;`).

Example:

```
List<Integer> nums = new ArrayList<>();  
nums.add(10);           // autobox int → Integer  
int first = nums.get(0); // unbox Integer → int
```

Q. Mutable vs Immutable. How do you write a custom immutable class?

A. Immutable = state can't change after construction. To make one:

1. Declare the class `final` (no subclass can break it).
2. All fields `private final`.
3. No setters.
4. Initialise everything in the constructor.
5. If a field is itself mutable (e.g., a `List`), return a defensive copy or an unmodifiable view.

Example:

```
public final class Address {  
    private final String city;  
    private final List<String> tags;  
  
    public Address(String city, List<String> tags) {  
        this.city = city;  
        this.tags = List.copyOf(tags); // defensive copy  
    }  
    public String getCity() { return city; }  
    public List<String> getTags() { return tags; } // already unmodifiable  
}
```

Q. Is Java pass-by-value or pass-by-reference?

A. Java is strictly **pass-by-value** — but for objects, the value being passed is the *reference*. So you can mutate the object's fields but you cannot make the caller's variable point to a new object.

Example:

```

void mutate(StringBuilder sb) { sb.append("X"); }
void reassign(StringBuilder sb) { sb = new StringBuilder("Y"); }

StringBuilder a = new StringBuilder("hello");
mutate(a);    System.out.println(a); // helloX
reassign(a);  System.out.println(a); // helloX – reassign didn't affect caller

```

17.6 Exception Handling

Q. What is the exception hierarchy?

A.

```

Throwable
├── Error          (OutOfMemoryError, StackOverflowError – don't catch)
├── Exception
│   ├── RuntimeException (unchecked: NPE, IllegalArgumentException, ClassCast...)
│   └── (other Exceptions) (checked: IOException, SQLException...)

```

Checked must be declared/caught at compile time. Unchecked don't have to be — but you can still catch them.

Q. Checked vs Unchecked — when do you create each?

A. Create a **checked** exception when the caller can reasonably recover (file missing → ask user). Create an **unchecked** exception when the caller can't recover meaningfully — typically a programming bug (invalid argument, illegal state).

Q. throw vs throws.

A. throw = actually throws an exception inside a method body. throws = declares in the method signature what exceptions can propagate out (required for checked).

Example:

```

public void readFile(String path) throws IOException {
    if (path == null) throw new IllegalArgumentException("path required");
    Files.readAllLines(Path.of(path));
}

```

Q. try / catch / finally / multi-catch / try-with-resources.

A.

```

try {
    // risky code
} catch (IOException | SQLException e) {    // multi-catch (Java 7+)

```

```

        // shared handling
    } finally {
        // always runs (close resources)
    }

    try (BufferedReader br = new BufferedReader(new FileReader(path));
        Connection cn = DriverManager.getConnection(url)) {
        // br and cn auto-closed in reverse order
    } catch (Exception e) { ... }

```

Q. Custom exception — how?

A.

```

public class PaymentDeclinedException extends RuntimeException {
    private final String reason;
    public PaymentDeclinedException(String reason, Throwable cause) {
        super("Payment declined: " + reason, cause);
        this.reason = reason;
    }
    public String getReason() { return reason; }
}

```

Extend `RuntimeException` for unchecked, `Exception` for checked. Always preserve the original cause via the constructor chain.

Q. Will `finally` always run?

A. Almost always — even on a thrown exception, even on a return inside the try. The exceptions are `System.exit()`, JVM crash, or a daemon thread being terminated. Don't return from `finally` — it silently overrides the value from try.

17.7 Collections Framework

Q. Give me an overview of the Collections framework.

A.

- **Collection** root: List, Set, Queue, Deque.
- **List** — ordered, duplicates allowed: ArrayList, LinkedList, Vector (legacy), Stack (legacy).
- **Set** — unique: HashSet, LinkedHashSet, TreeSet.
- **Queue/Deque** — FIFO/LIFO: ArrayDeque, PriorityQueue, LinkedList (also a Deque).
- **Map** (not a Collection): HashMap, LinkedHashMap, TreeMap, Hashtable (legacy), ConcurrentHashMap.

Q. ArrayList vs LinkedList — when do you use each?

A.

	ArrayList	LinkedList
--	-----------	------------

Backing	Dynamic array	Doubly-linked list
get(i)	O(1)	O(n)
add at end	O(1) amortised	O(1)
add/remove in middle	O(n) (shifting)	O(n) traversal + O(1) splice
Memory	Lower per element	Higher (prev/next pointers)
Cache locality	Good	Poor

Default to ArrayList. LinkedList wins only when you do many add/remove at both ends (it implements Deque).

Q. ArrayList vs Vector.

A. Vector is the legacy synchronised version of ArrayList. Every method is synchronised — slow and rarely what you want. For thread-safe lists today, use `Collections.synchronizedList(...)` or `CopyOnWriteArrayList`.

Q. How does HashMap work internally? (very common interview question)

A. A HashMap is an array of *buckets*. `put(key, value)` works like this:

1. Compute `key.hashCode()`.
2. Apply HashMap's internal hashing (XOR with high bits) to spread keys.
3. Determine the bucket index = `hash & (capacity - 1)`.
4. If the bucket is empty → store as a new node.
5. If there's a collision (same bucket) → traverse the bucket's linked list, compare keys with `equals()`. If found, replace value; else append.
6. Since Java 8, when a single bucket grows past 8 entries, the linked list is converted to a balanced **red-black tree** for O(log n) lookup.

Default initial capacity = 16, load factor = 0.75 → resize (doubling + rehash) when size exceeds 12. That's why `equals` + `hashCode` contract matters for custom keys.

Q. HashMap vs Hashtable vs ConcurrentHashMap.

A.

	HashMap	Hashtable	ConcurrentHashMap
Thread-safe	No	Yes (method-level sync)	Yes (segmented/CAS)
Performance under concurrency	N/A	Poor — single lock	Excellent
Null key/value	One null key, many null values	None allowed	None allowed
Legacy?	No	Yes	No

Use `HashMap` for single-threaded; `ConcurrentHashMap` for concurrent.

Q. HashSet vs TreeSet vs LinkedHashSet.

A. HashSet — unordered, $O(1)$ operations, no nulls? — allows one null. **TreeSet** — sorted (by natural order or `Comparator`), $O(\log n)$, no nulls. **LinkedHashSet** — preserves insertion order, $O(1)$, allows null.

Q. Comparable vs Comparator.

A. Comparable = natural ordering, implemented inside the class via `compareTo`. Use one.

Comparator = external ordering, often via lambda. Use many.

Example:

```
class Employee implements Comparable<Employee> {
    int id; String name; double salary;
    @Override public int compareTo(Employee o) { return this.id - o.id; }
}

Comparator<Employee> bySalary = Comparator.comparingDouble(e -> e.salary);
Comparator<Employee> byNameThenSalary =
    Comparator.comparing((Employee e) -> e.name).thenComparingDouble(e ->
e.salary);

list.sort(byNameThenSalary);
```

Q. Fail-fast vs Fail-safe iterators.

A. Fail-fast (most `java.util` collections): throw `ConcurrentModificationException` if the collection is structurally modified during iteration — including by another thread. **Fail-safe** (`CopyOnWriteArrayList`, `ConcurrentHashMap`): iterate over a snapshot, so they don't throw — but you may not see the latest changes.

Q. Iterator vs ListIterator.

A. `Iterator`: forward only, supports remove. `ListIterator` (`List` only): bidirectional, supports add/set/remove and lets you read index. Both are fail-fast.

17.8 Multi-threading basics

Q. Thread vs Runnable — which do you prefer?

A. Prefer `Runnable` (or `Callable<V>`) because Java has single inheritance; if you extend `Thread` you can't extend anything else. `Runnable` is also better for thread pools where you submit tasks to an executor.

Example:

```
Runnable task = () -> System.out.println("Hi from " +
Thread.currentThread().getName());
new Thread(task).start();

ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(task);
```

Q. Thread lifecycle states.

A. NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED. `start()` moves from NEW to RUNNABLE; the OS scheduler decides when it actually runs. `wait()` / `join()` / `sleep()` move it into waiting states.

Q. synchronized — what does it do?

A. Acquires a lock so only one thread at a time runs the synchronised block/method on the same object (instance methods lock this; static methods lock the Class object). Provides mutual exclusion and visibility (changes by the holding thread become visible to the next holder).

Example:

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }    // method lock
    public void decrement() {
        synchronized (this) { count--; }                // block lock
    }
}
```

Q. volatile — what does it do?

A. Guarantees visibility of writes to a variable across threads (no caching in CPU registers; reads/writes go through main memory). Doesn't provide atomicity for compound operations like `i++`. Use it for simple flags; use `AtomicInteger` or synchronisation for counters.

Q. wait, notify, notifyAll.

A. Used for inter-thread coordination on a shared lock. `wait()` — releases the lock and suspends until notified. `notify()` — wakes one waiting thread. `notifyAll()` — wakes all. Must be called inside a synchronised block on the same lock object.

Q. Difference between `sleep()` and `wait()`.

A. `Thread.sleep()` pauses the current thread for a duration; doesn't release any held lock. `Object.wait()` releases the lock on that object and waits for notify.

17.9 Java 8+ features

Q. List the Java 8 features you've used.

A.

- **Lambda expressions** — compact functional code.
- **Functional interfaces** — single-abstract-method targets for lambdas.
- **Streams API** — declarative data processing.
- **Method references** — `User::getEmail`.
- **Default and static methods** in interfaces.
- **Optional** — explicit nullability.
- **Date/Time API** — `java.time.LocalDate`, `LocalDateTime`, `Duration`.

Q. Lambda — syntax and rules.

A.

```
// no args
Runnable r = () -> System.out.println("hi");

// one arg (parens optional)
Function<String,Integer> len = s -> s.length();

// multiple args + block
BinaryOperator<Integer> add = (a, b) -> {
    int sum = a + b;
    return sum;
};
```

Lambdas can only target functional interfaces. Captured local variables must be effectively final.

Q. Method references — the four kinds.

A.

- **Static** — `Integer::parseInt`
- **Instance of a particular object** — `System.out::println`
- **Instance of an arbitrary object of a type** — `String::length`
- **Constructor** — `ArrayList::new`

Q. Streams — intermediate vs terminal operations.

A. Intermediate (lazy, return a `Stream`): `filter`, `map`, `flatMap`, `distinct`, `sorted`, `peek`, `limit`, `skip`.

Terminal (trigger execution): `forEach`, `collect`, `reduce`, `count`, `findFirst`, `findAny`, `anyMatch`, `allMatch`, `toArray`. A stream isn't executed until a terminal operation runs.

Example:

```
List<String> emails = users.stream()
    .filter(User::isActive)           // intermediate
    .map(User::getEmail)             // intermediate
    .sorted()                        // intermediate
```



```
.distinct() // intermediate
.collect(Collectors.toList()); // terminal
```

Q. map vs flatMap.

A. map: one-to-one — transforms each element. flatMap: one-to-many — each element produces a stream, all flattened into one.

Example:

```
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));
List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList()); // [1,2,3,4]
```

Q. What is Optional and why?

A. A container that may or may not hold a non-null value — used to make nullability explicit in the API and avoid NullPointerException. Methods: of, ofNullable, empty, isPresent, orElse, orElseThrow, map, flatMap.

Example:

```
public Optional<User> findByEmail(String email) { ... }

String name = findByEmail("a@b.com")
    .map(User::getName)
    .orElse("Anonymous");
```

Use it as a return type for methods that may not find a result. Don't use it for fields or method parameters.

Q. parallelStream() — when do you use it?

A. When the workload is genuinely CPU-heavy and the data set is large, and the operations are stateless and associative. For typical small or I/O-bound work it's usually slower than a sequential stream because of the fork-join overhead. Test before adopting.

Q. Common Collectors examples.

A.

```
// toList, toSet, toMap
List<String> names = users.stream().map(User::getName).collect(Collectors.toList());
Map<Integer,User> byId = users.stream().collect(Collectors.toMap(User::getId, u ->
u));

// groupingBy
```

```
Map<String, List<User>> byRole = users.stream()
    .collect(Collectors.groupingBy(User::getRole));

// counting per group
Map<String, Long> countByRole = users.stream()
    .collect(Collectors.groupingBy(User::getRole, Collectors.counting()));

// joining
String csv = names.stream().collect(Collectors.joining(", "));
```

Q. forEach vs traditional for loop.

A. forEach on a stream is declarative and reads cleanly, but you can't break out early easily. A traditional for loop gives you index access and early exit (break). Streams shine when you're chaining transformations; a plain for loop is often fine for simple iteration.

Q. What is the Java 8 Date/Time API and why was it introduced?

A. java.time replaces the legacy java.util.Date and Calendar, which were mutable, not thread-safe, and confusing. Key types: LocalDate, LocalTime, LocalDateTime, ZonedDateTime, Instant, Duration, Period. All immutable, thread-safe, with a fluent API.

Example:

```
LocalDate today = LocalDate.now();
LocalDate dob = LocalDate.of(1998, 6, 12);
int age = Period.between(dob, today).getYears();
String formatted = today.format(DateTimeFormatter.ofPattern("dd-MM-yyyy"));
```

21. Coding Interview Problems with Solutions

Q. Reverse a string (without `StringBuilder.reverse`).

A.

```
public static String reverse(String s) {
    char[] c = s.toCharArray();
    int i = 0, j = c.length - 1;
    while (i < j) {
        char t = c[i]; c[i] = c[j]; c[j] = t;
        i++; j--;
    }
    return new String(c);
}
```

Q. Check if a string is a palindrome.

A.

```
public static boolean isPalindrome(String s) {
    s = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    int i = 0, j = s.length() - 1;
    while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
    return true;
}
```

Q. Count vowels in a string.

A.

```
public static int countVowels(String s) {
    int count = 0;
    for (char c : s.toLowerCase().toCharArray())
        if ("aeiou".indexOf(c) >= 0) count++;
    return count;
}
```

Q. First non-repeating character.

A.

```
public static Character firstUnique(String s) {
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);
    return freq.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .findFirst()
        .map(Map.Entry::getKey)
        .orElse(null);
}
```

```
.map(Map.Entry::getKey)
    .findFirst().orElse(null);
}
```

Q. Check if two strings are anagrams.

A.

```
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    char[] x = a.toCharArray(); char[] y = b.toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

Q. Fibonacci — iterative.

A.

```
public static long fib(int n) {
    if (n < 2) return n;
    long a = 0, b = 1;
    for (int i = 2; i <= n; i++) { long t = a + b; a = b; b = t; }
    return b;
}
```

Q. Find duplicates in an array.

A.

```
public static Set<Integer> duplicates(int[] arr) {
    Set<Integer> seen = new HashSet<>(), dup = new HashSet<>();
    for (int n : arr) if (!seen.add(n)) dup.add(n);
    return dup;
}
```

Q. Find missing number in 1..n.

A.

```
public static int missing(int[] arr, int n) {
    int total = n * (n + 1) / 2;
    int sum = 0; for (int x : arr) sum += x;
    return total - sum;
}
```

Q. Reverse a number.

A.

```
public static int reverseNumber(int n) {
    int rev = 0;
    while (n != 0) { rev = rev * 10 + n % 10; n /= 10; }
    return rev;
}
```

Q. Check if a number is prime.

A.

```
public static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; (long) i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

Q. Swap two numbers without a third variable.

A.

```
a = a + b;
b = a - b;
a = a - b;
// or with XOR:
a = a ^ b; b = a ^ b; a = a ^ b;
```

Q. Count word occurrences in a string.

A.

```
public static Map<String,Long> wordCount(String s) {
    return Arrays.stream(s.toLowerCase().split("\\W+"))
        .filter(w -> !w.isEmpty())
        .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
}
```

Q. Remove duplicates from a List, preserving order.

A.

```
List<Integer> unique = list.stream().distinct().collect(Collectors.toList());
// or
LinkedHashSet<Integer> set = new LinkedHashSet<>(list);
```

Q. Sort a HashMap by value, descending.

A.

```
Map<String,Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey, Map.Entry::getValue,
        (a,b) -> a, LinkedHashMap::new));
```

Q. FizzBuzz.

A.

```
for (int i = 1; i <= 100; i++) {
    String s = "";
    if (i % 3 == 0) s += "Fizz";
    if (i % 5 == 0) s += "Buzz";
    System.out.println(s.isEmpty() ? i : s);
}
```

Q. Selenium coding — safe click with retry.

A.

```
public static void safeClick(WebDriver driver, By locator, int timeoutSec) {
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(timeoutSec));
    for (int i = 0; i < 3; i++) {
        try {
            wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
            return;
        } catch (StaleElementReferenceException | ElementClickInterceptedException e)
        {
            // retry
        }
    }
    throw new RuntimeException("Could not click " + locator + " after 3 attempts");
}
```

Q. Print 1–100 except multiples of 3 — using streams.

A.

```
IntStream.rangeClosed(1, 100)
    .filter(i -> i % 3 != 0)
    .forEach(System.out::println);
```

Q. Read a file line by line.

A.

```
try (Stream<String> lines = Files.lines(Paths.get("data.txt"))) {
    lines.forEach(System.out::println);
}
```


22. Scenario / Situational Testing Questions

Q. How would you test a pen?

A. First clarify: type (ball/gel/fountain), target user (adult/child/surgeon), environment (paper/whiteboard/zero-gravity). Then break it down: functional (writes, retracts, cap fits, refill works), non-functional (grip usability, reliability for 24h writing, drop durability), negative (empty refill, glass surface, extreme temperatures), safety (choking hazard cap, ink toxicity), compatibility (paper types, ink visibility).

Q. How would you test a login page?

A. Positive: valid credentials, "Remember me", case-sensitivity of email vs password.

Negative: wrong password, non-existent user, blank fields, SQL injection (' OR 1=1 --), XSS, paste of masked password.

Boundary: max-length email/password, leading/trailing spaces.

Security: account lockout, masked password, HTTPS only, no creds in URL, secure cookie flags, brute-force protection, password reset link expiry.

UX: tab order, Enter submits, error messages, accessibility.

Cross-browser/device: Chrome/Edge/Firefox/Safari/mobile.

Performance: 1000 concurrent logins.

Example: In one round I named all of the above in 60 seconds — interviewer stopped me at "security" and said "now design the test cases for security." Be ready for follow-ups on any category.

Q. How would you test an e-commerce checkout?

A. Cart correctness, price calculation (subtotal + tax + shipping – discount), coupon application/expiry/stacking, address validation, payment methods (card, UPI, COD, wallet), failure recovery (payment timeout, double-click), order confirmation email, inventory decrement, partial fulfilment, currency, mobile UX, accessibility, PCI security, performance under load, idempotency (no double-charge on retry).

Q. An automated test passes locally but fails in Jenkins. How do you debug?

A.

1. Read CI logs and any artifacts (screenshots, HARs, traces).
2. Compare environments — browser version, OS, screen size (headless defaults to 800×600), timezone, locale.
3. Check timing — CI is often slower; switch from implicit to explicit waits.
4. Verify test data — env-specific data missing.
5. Check parallelism — shared state, ports, file paths.
6. Try headless locally to reproduce.
7. If flaky, retry once with logging — and fix root cause.

Q. A bug only reproduces in production. What now?

A. Capture logs/traces/IDs, identify env-specific factors (data, scale, config, CDN cache), reproduce in a prod-like staging with PII-safe snapshot data, file with severity matching impact, suggest a workaround or feature flag, RCA — and add a test or monitor so it can't ship undetected again.

Q. Release is tomorrow and a P1 is found. What do you do?

A. Confirm reproducibility and impact, file P1, notify PO/EM/dev lead, evaluate options: hotfix, rollback, feature flag off, defer release. Help with cost/impact — how many users affected, is there a workaround. Don't decide alone — escalate with data.

Q. Developer says "works on my machine."

A. Share exact build/version, environment, steps with video, console + network logs, response payloads. Often it's a data difference, cache state, or feature-flag mismatch. Pair-debug if needed.

Q. You have 2 days for what needs 5. How do you prioritise?

A. Risk-based — critical user journeys + recent changes + historically defect-prone areas first. Cut nice-to-have polish tests. Communicate clearly to PO what's covered and what's deferred, with risk.

23. Behavioural & HR Questions

Use the **STAR** framework: **S**ituation → **T**ask → **A**ction → **R**esult. Keep answers ~90 seconds. Quantify outcomes where you can — % defect reduction, time saved, releases supported.

Q. Tell me about yourself.

A. Template (60–90s):

"I'm Vaishnavi, a QA Engineer with around 2 years of experience in [domain — e.g., e-commerce / fintech / SaaS]. I started in manual testing and have grown into automation — currently I work with Selenium + Java + TestNG, Postman/Rest Assured for API testing, and SQL for backend validation. In my current role at [company], I'm part of a Scrum team that releases every 2 weeks; I own end-to-end QA for [module], have built a regression automation suite that cut our cycle time from X days to Y hours, and have logged ~N defects of which Z were caught pre-production. Outside work I've been exploring AI-assisted testing tools like KaneAI/TestmuAI. I'm looking for a role where I can deepen my automation skills and take on more architectural ownership."

What they're really asking: Whether you can summarise yourself coherently and align the summary to the JD. They'll pick one thing you said and dig into it — so don't include anything you can't defend.

Q. What are your strengths?

A. "Three I'd highlight:

1. **Detail orientation** — I genuinely enjoy edge cases; in my last sprint I caught a date-rollover bug at year-end that no existing test covered.
2. **Communication with developers** — I write reproducible bug reports with logs and short videos, so reopens are rare.
3. **Continuous learning** — I taught myself Rest Assured for API automation in two weeks and brought it into our framework.

Each one has a specific example I can walk through."

Q. What is your weakness?

A. "In my first year I'd spend too long perfecting test cases before starting execution. I've moved to a 'just-enough' approach — outline cases, start exploratory testing early, refine as I learn. I still catch myself wanting to over-document, but I now timebox the planning phase."

Avoid: "I'm a perfectionist." It reads as canned. Pick a real, non-fatal weakness and show what you're doing about it.

Q. Why are you leaving your current company?

A. "I've had a good run there — shipped X, learned Y. I'm looking for a role with more automation depth and exposure to [scale / domain / tech from the JD]. From what I've read about your team's [tooling / product], it lines up well with where I want to grow."

Never badmouth a manager, peers, or the company. Frame it as moving *toward* something, not away from.

Q. Why this company?

A. Research beforehand. Mention: their product (a real feature you've explored), engineering culture (blog posts/open source), JD tech stack alignment, and role scope. Example: "I read your engineering blog post on chaos testing — that's the kind of QA maturity I want to be part of."

Q. Why should we hire you?

A. "For three reasons: I bring 2 years of hands-on QA across manual, automation, and API testing — so I can contribute on day one. I've built and maintained automation frameworks from scratch — relevant to your hybrid framework in the JD. And I'm comfortable working with developers and POs in Scrum, which from the JD seems to be how your team operates."

Q. Tell me about a challenging bug you found.

A. STAR example:

S: Intermittent failure on the order confirmation page — about 5% of users saw empty cart summaries despite successful payment.

T: Reproduce reliably and find the root cause.

A: Logs showed it cluster-by-cluster. I correlated with a cache TTL on order-service, reproduced by adding latency between payment and confirmation calls, worked with backend dev to add a synchronous fetch fallback.

R: Production complaints dropped to zero; added an automated API test that simulates the race condition and a Grafana alert on the cache miss ratio.

Q. Tell me about a time you disagreed with a developer.

A. "In a release-week sprint, a developer marked a defect as 'not reproducible' and closed it. I disagreed — I had a video and HAR. Instead of reopening silently, I set up a 15-minute screen-share. We walked through the steps together and found it only reproduced for users with a specific feature flag enabled. We reopened it, fixed it before release. Lesson: when you disagree, bring evidence and collaborate, don't push back in comments."

Q. Tell me about a time you worked under pressure.

A. Real, recent example — hotfix during a release weekend, P1 found late, owning regression for a critical feature with tight scope. Describe how you prioritised, communicated, and what the outcome was.

Q. Tell me about a time you failed.

A. "In my first six months I missed a defect that reached production — a currency-formatting bug that displayed ₹ as \$ for Indian users on iOS. I'd validated the API but not the rendered string on the device. Lesson: I started keeping a personal 'defect taxonomy' and added rendered-output checks to my mobile test plan. Haven't missed a similar one since."

Q. How do you handle conflict in a team?

A. "Privately first, with curiosity not blame — 'help me understand why you see it that way.' Almost always there's missing context. If we still disagree I'll bring data and loop in the EM/PO for a decision. Goal is the product, not who's right."

Q. Where do you see yourself in 5 years?

A. "I'd like to grow into an SDET / QA architect role — designing test strategy, mentoring juniors, contributing to engineering decisions on testability. Concretely in the next 2 years I want to deepen performance and security testing, and in 5 years I'd be comfortable owning QA for a product end-to-end."

Q. How do you handle criticism?

A. "I separate the feedback from the delivery — focus on the content. I'll ask one clarifying question, take notes, thank the person, and try the change. In retros I now actively ask my team 'what should I do differently next sprint?' — the answers have made me much faster at writing automation."

Q. Manager assigns you a task you don't know how to do.

A. "I'd timebox 1–2 hours of self-research — docs, similar code in our repo, official tutorials. If still stuck, I'd write down what I've tried and ask a peer or my manager a specific question, not a vague one. Then I'd block half an hour daily for the next few days to skill up so I can own similar work next time."

Q. Why QA and not development?

A. "Honestly because I find breaking systems more interesting than building them. Good QA needs both — I read and write code, I just direct it at finding gaps rather than adding features. I also like the cross-functional nature: I work with devs, BAs, POs, support — I see the whole product."

Q. Tell me about a time you took initiative.

A. Real example — introduced API automation, started a weekly 'bug bash', built a defect-metrics dashboard, mentored an intern. Focus on action and impact, not just intent.

24. Salary, Notice Period & Negotiation

Q. What is your current CTC?

A. State fixed + variable explicitly. "My current fixed is ₹X.X LPA with a variable of up to ₹Y, taking total CTC to ₹Z LPA." Don't inflate — it's verifiable from your offer letter and Form 16.

Q. What is your expected CTC?

A. "Based on my 2 years of experience, my skill mix (Selenium, API, SQL, plus exposure to AI-assisted testing and mobile via Appium), and the market for similar roles in [city], I'm looking at a hike of around 30–40% on my current — so approximately ₹X.X LPA fixed. I'm open to discussing the structure if the role and growth align."

Research Glassdoor, AmbitionBox, LinkedIn Salary, and friends in the role. Give a range, anchor at the higher end. Never go below your floor.

Q. What is your notice period? Can it be reduced?

A. "My official notice period is X days. I can try to negotiate an early release, and buy-out is also an option depending on the offer." Don't promise a date you can't hit.

Q. Do you have any other offers?

A. Be honest. If yes: "I'm in final rounds with one other company; happy to share more if helpful." Don't bluff — it usually backfires.

Q. When can you join?

A. Realistic notice period plus a small buffer. "Best case 30 days post offer; worst case 60. I'll confirm once I formally resign."

25. Questions YOU Should Ask the Interviewer

Always ask 2–3 thoughtful questions. It signals interest and helps your decision.

- **To the manager:** "How is success measured for someone in this role at the 3-, 6-, and 12-month marks?"
- **To the manager:** "What does a typical sprint look like for QA? How are you involved at planning?"
- **To the QA lead:** "What's the current state of your automation suite — coverage, flakiness, ownership model?"
- **To the team:** "How are production defects handled — RCAs, blameless post-mortems?"
- **To the team:** "What's the biggest quality challenge the team is solving right now?"
- **To engineering:** "How do dev and QA collaborate on testability — are devs writing unit/integration tests?"
- **Career:** "What does growth look like for an SDET at your company? Are there clear levels?"
- **Tooling:** "What's the CI/CD pipeline like? How often do you release?"
- **Culture:** "What do people who thrive on your team have in common?"
- **HR:** "What's the next step in the process and when can I expect to hear back?"

26. Last-Mile Checklist

One week before

- Re-read your own resume — be able to defend every line.
- Pick 3 projects you'll talk about; prepare STAR stories for each.
- Memorise key metrics: defects logged, automation coverage %, releases supported.
- Run your automation framework locally; have a code sample on GitHub ready.
- Practise coding problems on paper / whiteboard — at least 10 from this doc.
- Research the company: product, blog, recent news, competitors.

One day before

- Re-read JD; note 5 keywords and have a story for each.
- Test camera, mic, internet, lighting (if virtual).
- Notepad & pen, water, charger ready.
- Sleep early.

On the day

- Log in 5–10 minutes early.
- Greet warmly, smile, breathe.
- For coding: *think aloud*, clarify input/output, write brute force first, then optimise.
- If you don't know something: "I haven't worked with X directly, but based on Y I'd approach it like..." — never bluff a fact.
- End by asking your prepared questions.
- Thank-you note within 24h.

Topic-by-topic last-minute revision (one-liners)

- **SDLC/STLC** — sequential phases; STLC parallels SDLC for testing.
- **Test pyramid** — many unit, few UI; opposite is "ice-cream cone."
- **Severity vs priority** — technical impact vs business urgency.
- **Smoke / sanity / regression** — stable? feature works? old still works?
- **Selenium 4** — W3C, relative locators, Selenium Manager, CDP.
- **Locators** — id > name > css > xpath.
- **Waits** — explicit > implicit; never mix.
- **POM** — one class per page, locators + actions.
- **TestNG order** — Suite → Test → Class → Method → @Test.
- **BDD** — Gherkin, hooks, tags, runners.
- **API status codes** — 2xx OK, 3xx redirect, 4xx client, 5xx server.

- **Idempotency** — GET/PUT/DELETE yes, POST/PATCH no.
- **SQL** — joins, group by, having, window functions, indexes, ACID.
- **Mobile** — native/hybrid/web; emulator vs real; interrupts, network, orientation, permissions.
- **Appium** — UIAutomator2/XCUITest, accessibility id, hybrid context switching.
- **JMeter** — Thread Group, Samplers, Assertions, Listeners; percentiles > average.
- **Playwright** — auto-wait, role-based locators, trace viewer, contexts vs handles.
- **AI in QA** — generation, self-healing, visual, prompt engineering; always verify.
- **Java** — OOP pillars, collections, equals + hashCode, exceptions, streams.
- **STAR** — Situation, Task, Action, Result for every behavioural answer.

Final note: Interviewers hire people they want to work with. Be honest about what you don't know, curious about their problems, and clear about what you've actually done. You don't need to be perfect — you need to be coherent, calm, and credible.

— End of guide v2 — Best of luck for your interviews!